

Συστήματα Παράλληλης Επεξεργασίας

1η Εργαστηριακή Άσκηση

Κακούρης Δημήτριος el19019

Κλήμου Ανθή el19033

Κοσμάς Θωμάς el19845

1.2 Conway's Game of Life

Προκειμένου να παραλληλοποιήσουμε το Game Of Life εντοπίζουμε τα σημεία του κώδικα τα οποία επιδέχονται παραλληλοποίηση. Το Game Of Life υπολογίζει το state του κάθε κελιού του grid σύμφωνα με τους κανόνες που δίνονται στην εκφώνηση. Κρατάμε δυο states το προηγούμενο και το τωρινό grid, το προηγούμενο αφορά τη στιγμή που απεικονίζουμε το grid ώστε να υπολογίσουμε το τωρινό, δηλαδή το τωρινό αφορά την $t+1$ στιγμή και το προηγούμενο την t .

Προκειμένου να κάνουμε χρήση του API της OpenMP, κάνουμε `#include <omp.h>` και μεταγλωττίζουμε κατάλληλα το πρόγραμμα με το flag `-fopenmp`.

Παραθέτουμε το `make_on_queue.sh` το οποίο χρησιμοποιούμε για να υποβάλλουμε στη συστοιχία το job που αφορά το make:

```
parlab09@scirouter:~/ex1_dimitris$ cat make_on_queue.sh
#!/bin/bash

## Give the Job a descriptive name
#PBS -N makejob

## Output and error files
#PBS -o makejob.out
#PBS -e makejob.err

## How many machines should we get?
#PBS -l nodes=1

## Start
## Run make in the src folder (modify properly)

mkdir -p ${HOME}/tmp
export TMPDIR=${HOME}/tmp

module load openmp
cd /home/parallel/parlab09/ex1_dimitris/
make

rm -r ${HOME}/tmp
```

Παραθέτουμε το run_on_queue.sh το οποίο χρησιμοποιούμε για να υποβάλλουμε στη συστοιχία το job που αφορά την εκτέλεση:
Εδώ βλέπουμε συγκεκριμένα το bash script για 64x64 grid, έχουμε φτιάξει άλλα δυο για 1024x1024 και 4096x4096.

```
#!/bin/bash

## Give the Job a descriptive name
#PBS -N run_omp_game

## Output and error files
#PBS -o run_omp_game_64.out
#PBS -e run_omp_game_64.err

## How many machines should we get?
#PBS -l nodes=1:ppn=8

##How long should the job run for?
#PBS -l walltime=00:05:00

## Start
## Run make in the src folder (modify properly)

module load openmp
cd /home/parallel/parlab09/ex1_dimitris

for threads in 1 2 4 6 8
do
module load openmp
export OMP_NUM_THREADS=${threads}
./Game_of_Life 64 1000
done
```

Επιλέγουμε να παραλληλοποιήσουμε το πρώτο loop δηλαδή το loop που αφορά τις γραμμές του grid.

Επιλέγουμε να κάνουμε private τις μεταβλητές nbrs, i, j. Διαφορετικά θα οδηγηθούμε σε race conditions.

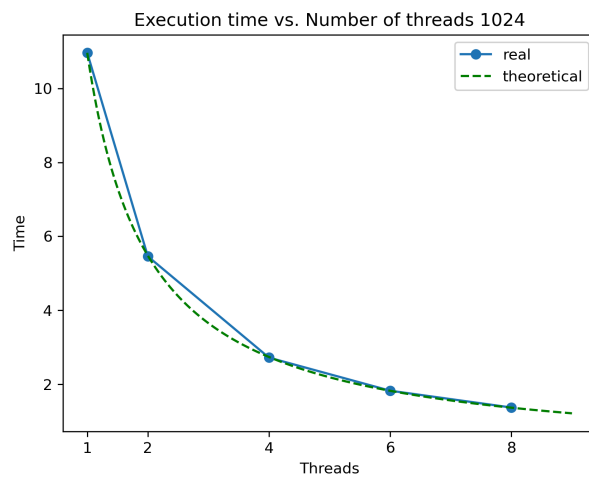
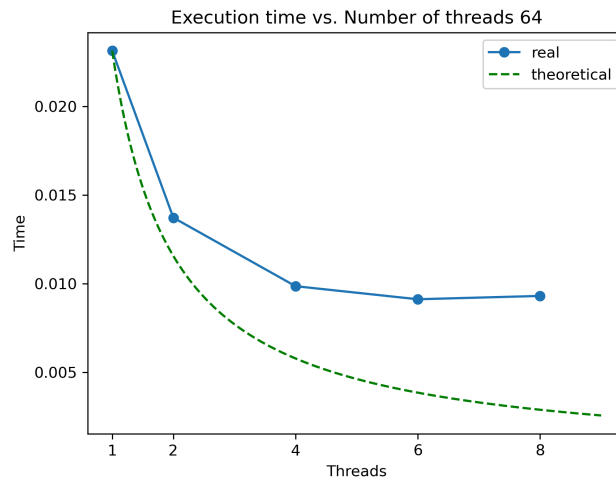
Γράφουμε επίσης ρητά ότι θέλουμε να είναι shared το previous και current state.

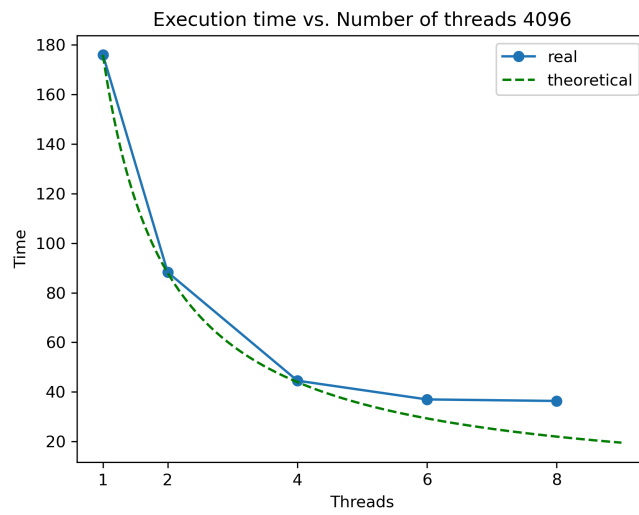
```
/*Game of Life*/

gettimeofday(&ts,NULL);
for ( t = 0 ; t < T ; t++ ) {
    #pragma omp parallel for private(nbrs,i,j) shared (previous,current)
    for ( i = 1 ; i < N-1 ; i++ )
        for ( j = 1 ; j < N-1 ; j++ ) {
            nbrs = previous[i+1][j+1] + previous[i+1][j] + previous[i+1][j-1] \
                + previous[i][j-1] + previous[i][j+1] \
                + previous[i-1][j-1] + previous[i-1][j] + previous[i-1][j+1];
            if ( nbrs == 3 || ( previous[i][j]+nbrs ==3 ) )
                current[i][j]=1;
            else
                current[i][j]=0;
        }
}
```

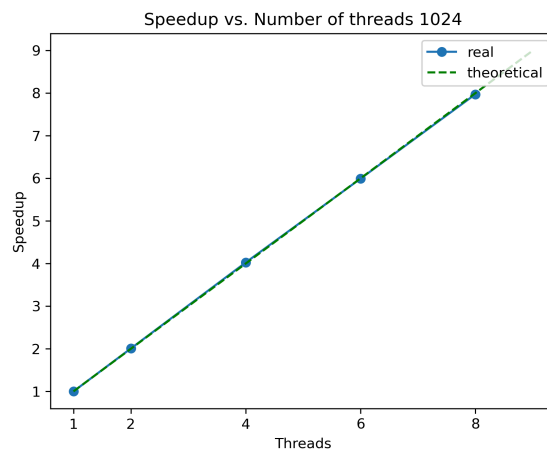
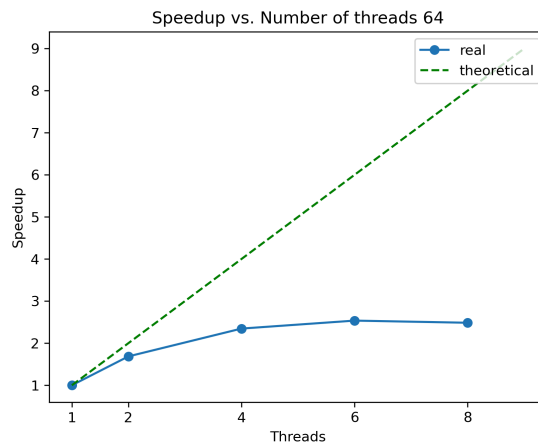
Εκτελούμε για 1000 εποχές το Game Of Life για 1,2,4,6,8 cores και μεγέθη ταμπλώ 64×64, 1024×1024 και 4096×4096.

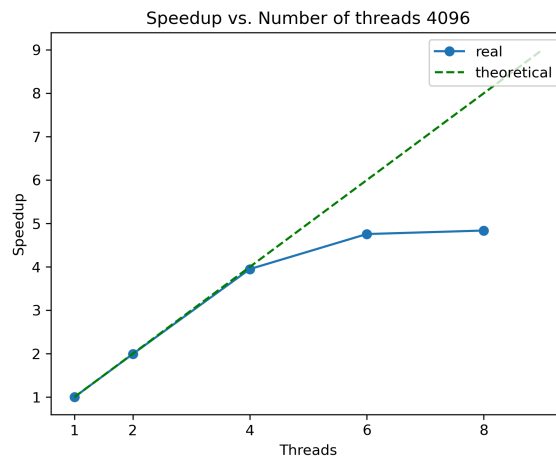
Έτσι παίρνουμε τα παρακάτω διαγράμματα χρόνου εκτέλεσης:





Αντίστοιχα τα διαγράμματα Speedup:





Παρατηρήσεις:

Στην ιδεατή περίπτωση θα περιμέναμε με αύξηση των threads, λόγω της δυνατότητας παραλληλοποίησης που μας προσφέρει η OpenMP, να οδηγεί σε αντίστοιχο speedup του χρόνου εκτέλεσης

Παρατηρούμε πώς το linear speedup το έχουμε στη περίπτωση του grid size 1024 x 1024. Στις άλλες περιπτώσεις:

- Περίπτωση 64 x 64:
Στη περίπτωση αυτή η προσθήκη threads δε κλιμακώνει γραμμικά, είναι μικρό το μέγεθος του grid και το overhead που μας δίνει το communication των threads δυσχεραίνει τη κλιμάκωση.
- Περίπτωση 4096 x 4096:
Στη περίπτωση πάλι δεν έχουμε γραμμική κλιμάκωση, είναι μεγάλο το μέγεθος του grid η συνεχής πρόσβαση στη μνήμη δυσχεραίνει τη κλιμάκωση, λόγω μη αποδοτικής χρήσης της cache. Παρατηρούμε εδώ πιο έντονα το γεγονός ότι το Game Of Life είναι memory bound.

2.1 Παραλληλοποίηση και βελτιστοποίηση του αλγορίθμου K-means

Shared Clusters:

Η πρώτη υλοποίηση που καλούμαστε να συμπληρώσουμε είναι η naive υλοποίηση. Ζητείται το configuration:

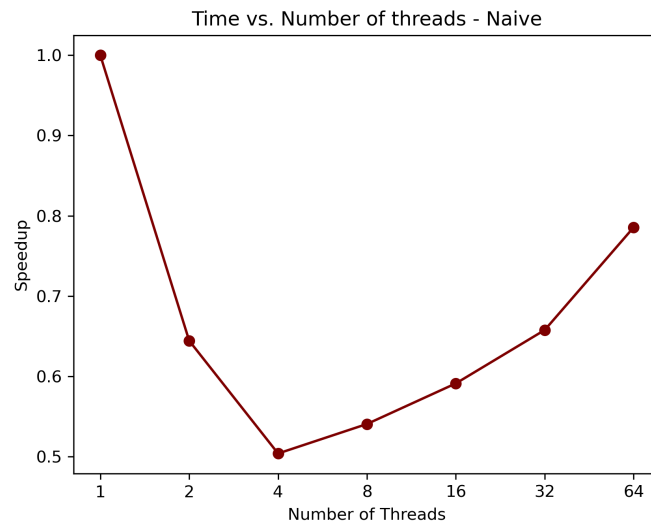
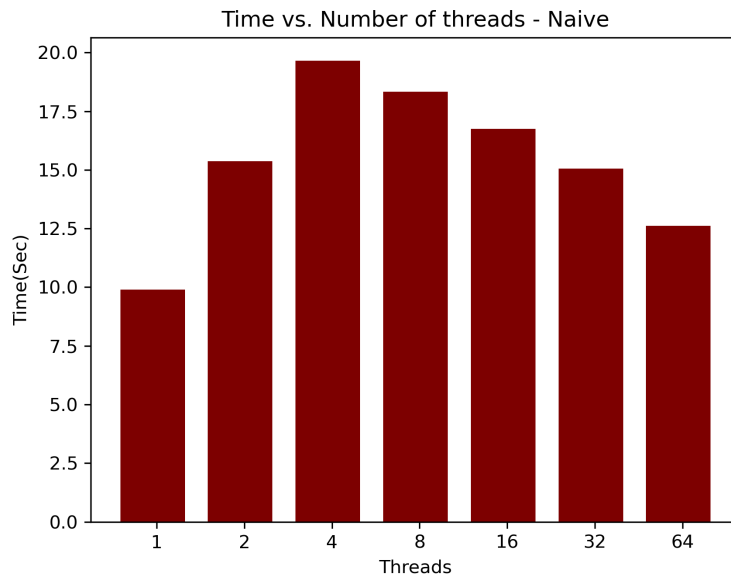
`{Size, Coords, Clusters, Loops} = {256, 16, 16, 10}`

Καλούμαστε να παραλληλοποιήσουμε το πιο υπολογιστικά βαρύ κομμάτι του κώδικα που είναι η εύρεση του κοντινότερου cluster για κάθε σημείο του dataset. Ο υπολογισμός γίνεται με τη συνάρτηση `find_nearest_cluster()`.

```
29  */
30
31  #pragma omp parallel for shared(newClusters, objects, membership, clusters, newClusterSize, numCoords, delta) private(i, j, index)
32  for (i=0; i<numObjs; i++) {
33      // find the array index of nearest cluster center
34      index = find_nearest_cluster(numClusters, numCoords, 6objects[i*numCoords], clusters);
35
36      // if membership changes, increase delta by 1
37      if (membership[i] != index)
38          #pragma omp atomic
39          delta += 1.0;
40
41      // assign the membership to object i
42      membership[i] = index;
43
44      // update new cluster centers : sum of objects located within
45      /*
46      * TODO: protect update on shared "newClusterSize" array
47      */
48      #pragma omp atomic
49      newClusterSize[index]++;
50
51
52      for (j=0; j<numCoords; j++)
53          /*
54          * TODO: protect update on shared "newClusters" array
55          */
56          #pragma omp atomic
57          newClusters[index*numCoords + j] += objects[i*numCoords + j];
58  }
```

Στη παραπάνω υλοποίηση παραλληλοποιούμε το loop με `#pragma omp parallel for`. Κλειδώνουμε τη με ατομική εντολή τη μεταβλητή `delta` όταν τα threads επιθυμούν να αυξήσουν τον αριθμό του count των αλλαγών στο `membership`, ενώ κλειδώνουμε με ατομική εντολή και το πίνακα μεγέθους των clusters, το `newClusterSize` και τον πίνακα που έχει τα κέντρα των clusters, το `newClusters`. Η ατομική εντολή προσδιορίζεται από το `#pragma omp atomic`. Παρατηρούμε στο διάγραμμα χρόνου εκτέλεσης προς τον αριθμό των νημάτων τα παρακάτω αποτελέσματα. Για τα benchmarks χρησιμοποιούμε το μηχανήμα `sandman` που μας δίνει τη δυνατότητα να χρησιμοποιήσουμε 64 threads.

Παρατηρούμε πως φαίνεται η σειριακή υλοποίηση είναι η γρηγορότερη εκ των υπολοίπων, αυτό οφείλεται στο ανταγωνισμό για το lock των atomic instructions που έχουμε στο κώδικα, συνολικά γίνονται πολλές προσβάσεις στους πίνακες `newClusters` και `newClusterSize`. Όταν κλειδώνει η πρόσβαση από ένα νήμα, κλειδώνει όλος ο πίνακας για τα άλλα νήματα οπότε έχουμε συνωστισμό. Επίσης ο χρόνος εκτέλεσης χειροτερεύει και από φαινόμενα όπως ότι τα δεδομένα τα οποία θέλει ο κάθε επεξεργαστής δεν είναι pinned στη cache αυτού.

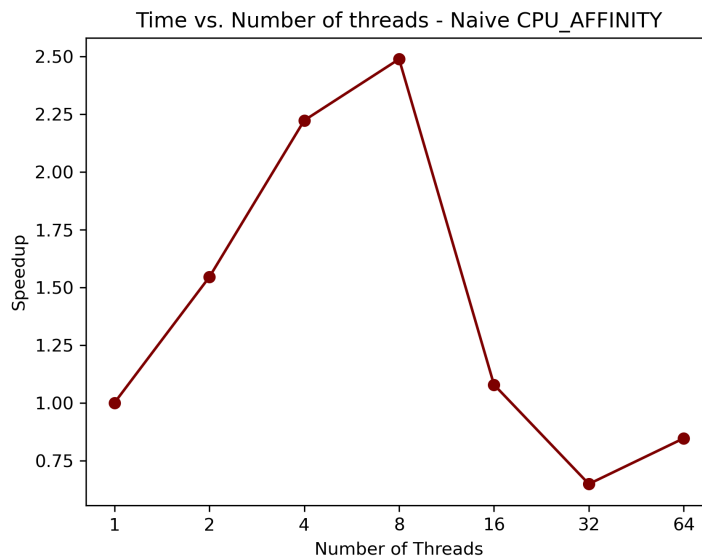
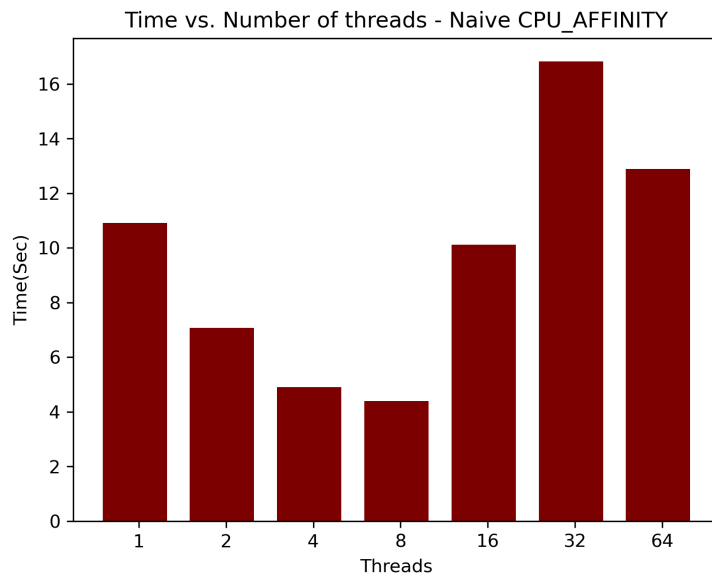


Το ζήτημα του pinning των δεδομένων το αντιμετωπίζουμε με την επιλογή GOMP_CPU_AFFINITY:

```
for i in 1 2 4 8 16 32 64
do
    export OMP_NUM_THREADS=${i}
    export GOMP_CPU_AFFINITY="0-63"
```

Έτσι προσδένουμε τα δεδομένα του κάθε επεξεργαστή στη cache του, βελτιώνοντας το locality των δεδομένων. Συγκεκριμένα στους επεξεργαστές 0 έως 31. Παρατηρούμε όπως είναι αναμενόμενο καλύτερους χρόνους και βελτίωση σε χρόνο στις παράλληλες

υλοποιήσεις σε σχέση με τη σειριακή. Χάνονται αυτά τα πλεονεκτήματα απο 16 threads και μετά λόγω αυξημένου contention. Καλύτερη επίδοση παρατηρούμε στα 8 threads.



Copied and Reduced Clusters:

Ορίζουμε τα τοπικά clusters που θα χρησιμοποιηθούν για να μαζέψουν τα επιμέρους αριθμητικά αποτελέσματα για την ανανέωση των “γενικών” clusters, όπως φαίνεται στο

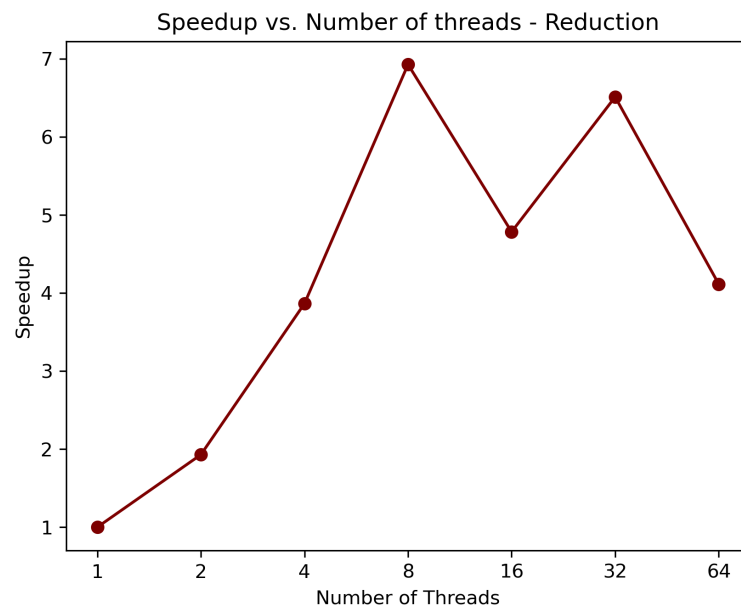
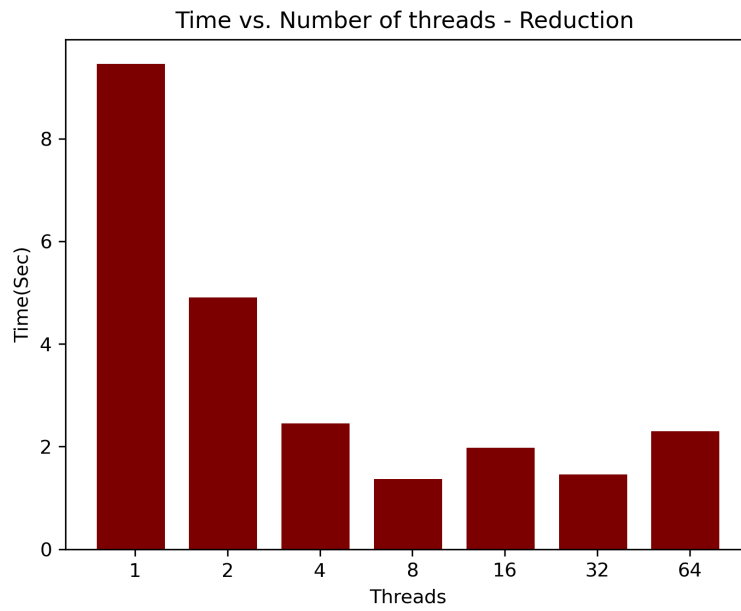
πρώτο TODO. Στη συνέχεια υλοποιούμε τη παραλληλοποίηση της εύρεσης των κοντινότερων clusters.

```
94 /*
95  * TODO: Initilize local cluster data to zero (separate for each thread)
96  */
97 for(k=0; k<nthreads; k++){
98     for (i=0; i<numClusters; i++) {
99         for (j=0; j<numCoords; j++)
100             local_newClusters[k][i*numCoords + j] = 0.0;
101         local_newClusterSize[k][i] = 0;
102     }
103 }
104 #pragma omp parallel shared(local_newClusters, objects,delta,membership,clusters,local_newClusterSize,numCoords) private(i,j,index)
105 {
106     #pragma omp for
107     for (i=0; i<numObjs; i++)
108     {
109         // find the array index of nearest cluster center
110         index = find_nearest_cluster(numClusters, numCoords, &objects[i*numCoords], clusters);
111
112         // if membership changes, increase delta by 1
113         if (membership[i] != index)
114             #pragma omp atomic
115             delta += 1.0;
116
117         // assign the membership to object i
118         membership[i] = index;
119     }
120 }
```

Αντι να έχουμε ένα global array για clusters όπως στη naive υλοποίηση και να δημιουργείται ένα bottleneck λόγω των προσβάσεων που επιθυμούν να πραγματοποιήσουν τα threads και να ανανεώσουν τόσο το newClusters όσο και το newClusterSize καθώς τότε κλειδώνεται όλος ο πίνακας τώρα το κάθε thread γράφει σε δικά του τοπικά arrays και το master thread στο τέλος μαζεύει αποτελέσματα με #pragma omp master. Έτσι προκύπτει και η υλοποίηση του reduction.

```
59 // update new cluster centers : sum of all objects located within (average will be performed later)
60 /*
61  * TODO: Collect cluster data in local arrays (local to each thread)
62  * Replace global arrays with local per-thread
63  */
64 int tid = omp_get_thread_num();
65 local_newClusterSize[tid][index]++;
66 for (j=0; j<numCoords; j++)
67     local_newClusters[tid][index*numCoords + j] += objects[i*numCoords + j];
68 }
69
70 /*
71  * TODO: Reduction of cluster data from local arrays to shared.
72  * This operation will be performed by one thread
73  */
74 #pragma omp barrier
75 #pragma omp master
76 {
77     for (k=1; k<nthreads; k++)
78     {
79         for (i=0; i<numClusters; i++)
80         {
81             for (j=0; j<numCoords; j++) {
82                 local_newClusters[0][i*numCoords + j] += local_newClusters[k][i*numCoords + j];
83             }
84             local_newClusterSize[0][i] += local_newClusterSize[k][i];
85         }
86     }
87 }
88 }
```

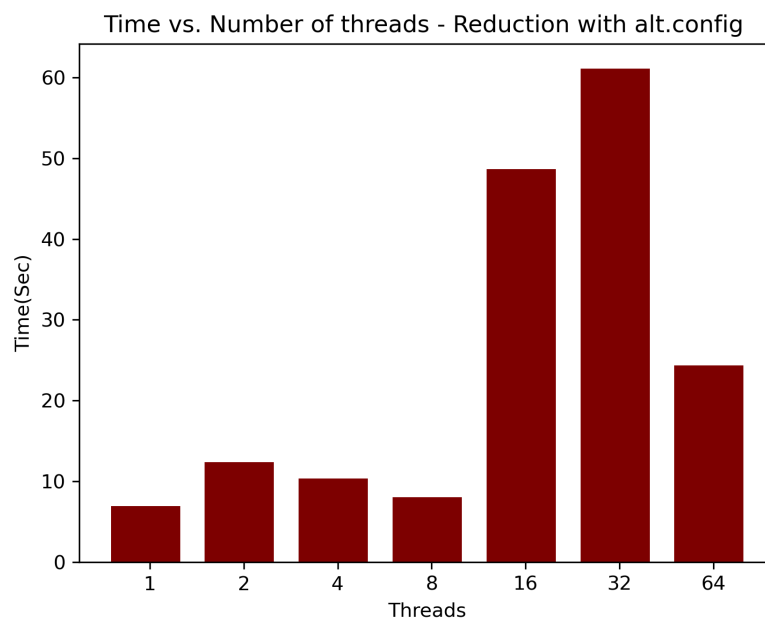
Στο reduction implementation έχουμε σχεδόν γραμμικό speedup μέχρι και 8 threads, από 16 και μετά δεν παρατηρείται βελτίωση στο χρόνο (ακόμα υπάρχει και χειροτέρευση για 64 threads).

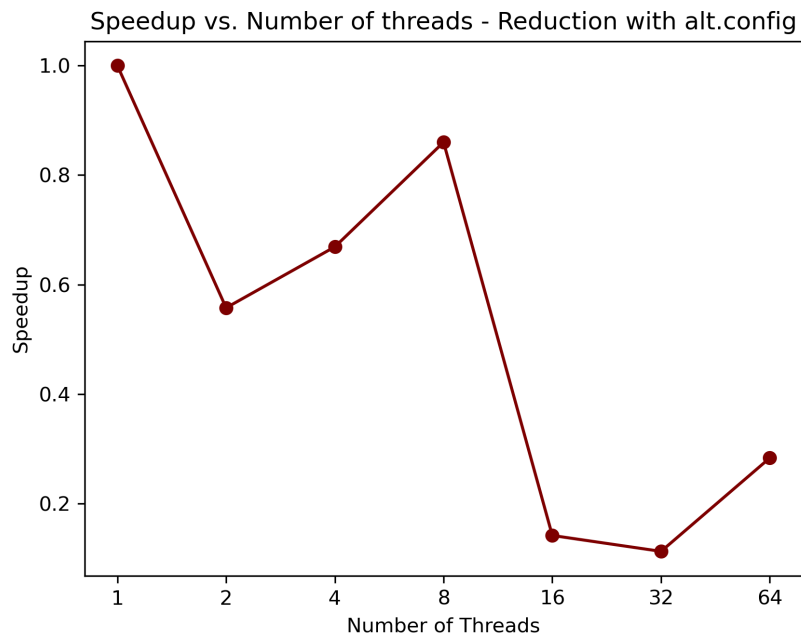


Ζητείται να συγκρίνουμε και με μια εναλλακτική αρχικοποίηση:

{Size, Coords, Clusters, Loops} = {256, 1, 4, 10}

Παρατηρούμε στη παραμετροποίηση αυτή πως ο καλύτερος χρόνος προκύπτει απο τη σειριακή υλοποίηση. Διαπιστώνουμε ότι οι χρόνοι εκτέλεσης και η κλιμάκωση δεν είναι ικανοποιητική. Αυτό οφείλεται στον μικρό αριθμό των clusters, ο οποίος οδηγεί στο να βρίσκονται περισσότερες γραμμές των πινάκων `local_newClusters` και `local_newClusterSize` στην ίδια περιοχή της cache μνήμης. Αποτέλεσμα αυτού είναι το φαινόμενο του False Sharing, κατά το οποίο η εγγραφή δεδομένων από ένα νήμα συμπίπτει χωρικά με αυτές των άλλων νημάτων, καθώς γράφονται στην ίδια cache line. Σε επόμενη εγγραφή τα νήματα πρέπει να κάνουν invalidate τη cache line που θέλουν να γράψουν και ύστερα να φορτώσουν εκ νέου τα δικά τους δεδομένα. Όλο αυτο δημιουργεί αργοπορία λόγω της μετακίνησης δεδομένων από κύρια μνήμη και ανάποδα, που χειροτερεύει με τον αριθμό των νημάτων. Ο καλύτερος χρόνος που πετύχαμε είναι 6.9 sec στη σειριακή υλοποίηση.





Για να αποφύγουμε αυτό το φαινόμενο, θα μπορούσαμε να προσθέσουμε μηδενικά στο τέλος κάθε τμήματος των πινάκων που αντιστοιχεί σε κάποιο thread (padding), ώστε να συμπληρωθεί το επιθυμητό μήκος cache line. Έτσι, αποφεύγουμε να επεξεργάζονται το ίδιο cache line δύο ή περισσότερα threads.

Η πολιτική "first touch" στα συστήματα NUMA ορίζει ότι η μνήμη που ανατίθεται σε ένα νήμα προέρχεται από την περιοχή μνήμης που είναι πιο κοντά στον επεξεργαστή που εκτελεί αυτό το νήμα για πρώτη φορά. Αυτό βελτιστοποιεί την απόδοση, καθώς ελαχιστοποιείται η καθυστέρηση πρόσβασης στη μνήμη. Έχουμε θέσει όπως είχε ζητηθεί `GOMP_CPU_AFFINITY='0-63'` για τα ερωτήματα του reduction και έτσι τα νήματα προσδένονται στις CPU που πρωτοέφεραν τα δεδομένα τους. Τώρα όλα τα νήματα που θα χρειάζονται το δεδομένο αυτό δε θα το πηγαινοφέρνουν μεταξύ της RAM των nodes που εκείνα βρίσκονται και αυτής της αρχικής ανάθεσης.

Το μεγαλύτερο ίσως optimization είναι η παραλληλοποίηση του allocation των `local_newClusters` και `local_newClusterSize`, έτσι το κάθε νήμα κάνει allocate blocks με private τρόπο, επομένως μειώνεται αρκετά η πιθανότητα να πέσουν blocks από δύο ξεχωριστά

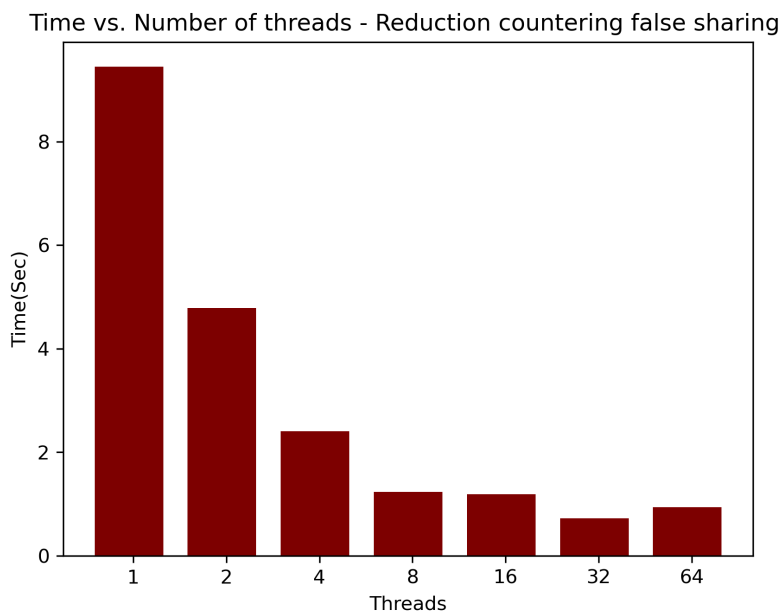
νήματα στην ίδια Cache Line. Τα παραπάνω τα υλοποιούμε με `#pragma omp parallel for private (k)`. Δε χρειάζεται να κάνουμε padding μιας και κάνουμε παραλληλοποίηση του allocation.

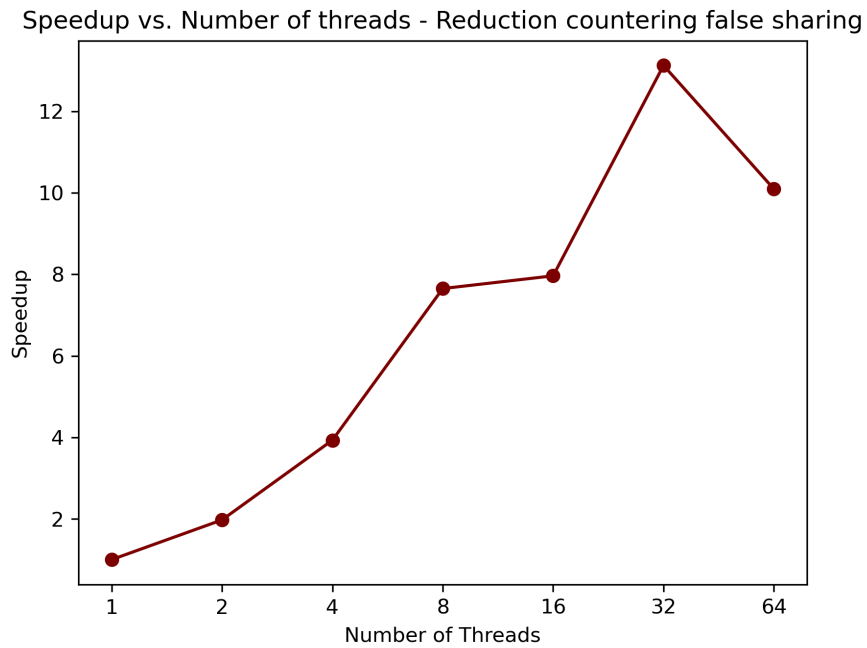
```
// initialize newClusterSize and newClusters to all 0
newClusterSize = (typeof(newClusterSize)) calloc(numClusters, sizeof(*newClusterSize));
newClusters = (typeof(newClusters)) calloc(numClusters * numCoords, sizeof(*newClusters));

// Each thread calculates new centers using a private space. After that, thread 0 does an array reduction on them.
int * local_newClusterSize[nthreads]; // [nthreads][numClusters]
double * local_newClusters[nthreads]; // [nthreads][numClusters][numCoords]

/*
 * Hint for false-sharing
 * This is noticed when numCoords is low (and neighboring local_newClusters exist close to each other).
 * Allocate local cluster data with a "first-touch" policy.
 */
// Initialize local (per-thread) arrays (and later collect result on global arrays)
#pragma omp parallel for private (k)
for (k=0; k<nthreads; k++)
{
    local_newClusterSize[k] = (typeof(*local_newClusterSize)) calloc(numClusters, sizeof(**local_newClusterSize));
    local_newClusters[k] = (typeof(*local_newClusters)) calloc(numClusters*numCoords, sizeof(**local_newClusters));
}
```

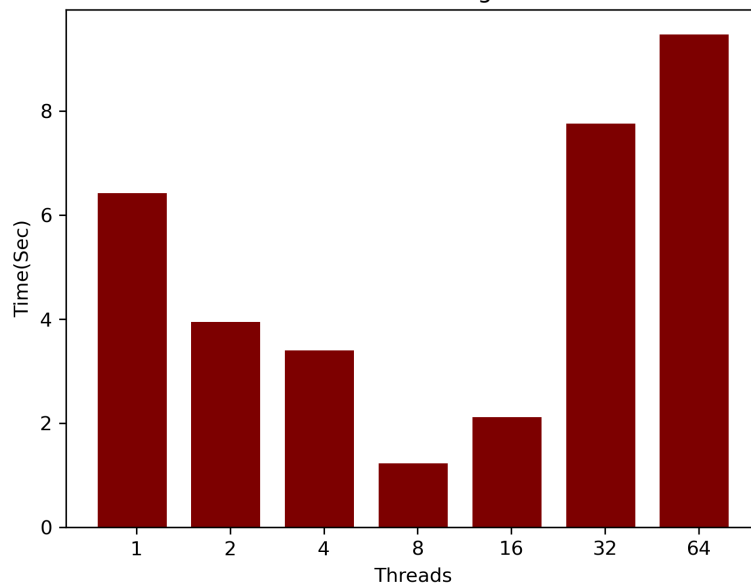
Παρατηρούμε πως και στις δύο περιπτώσεις έχουμε αισθητά βελτιωμένο χρόνο εκτέλεσης:



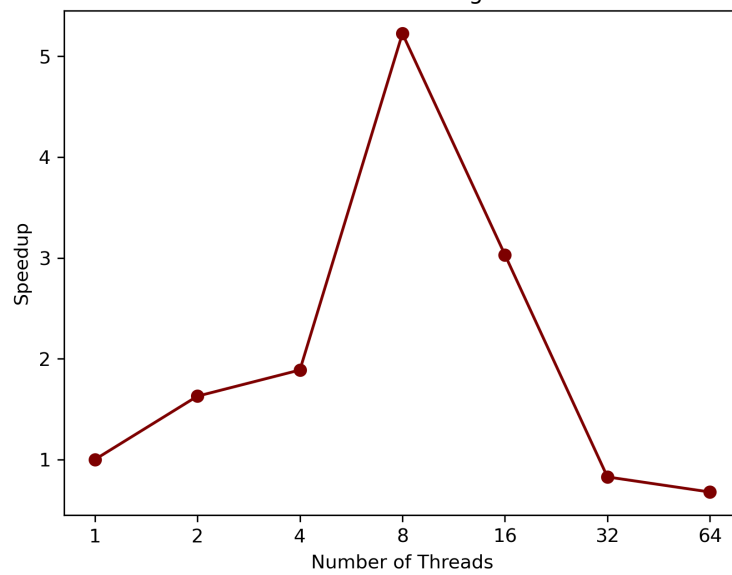


Ειδικά στη περίπτωση του alternative configuration επειδή το μέγεθος της cache line είναι 64 bytes, συμπεραίνουμε πως ο πίνακας των localClusters για ένα νήμα χωράει ακριβώς στη μισή cache line. Με συνέπεια το ακριβώς διπλανό νήμα να φέρνει το δικό του localClusters στην ίδια γραμμή κάνοντας συνεχώς invalidate τα άλλα αντίγραφα των cache lines και ξαναφορτώνοντας με τα δικά του δεδομένα. Με το padding όμως αναγκάζουμε κάθε επόμενο localClusters να πάει στην απο κάτω cache line και η διαφορά είναι εμφανής.

Time vs. Number of threads - Reduction countering false sharing with alt.config



Speedup vs. Number of threads - Reduction countering false sharing with alt.config



2.2 Παραλληλοποίηση του αλγορίθμου Floyd-Warshall

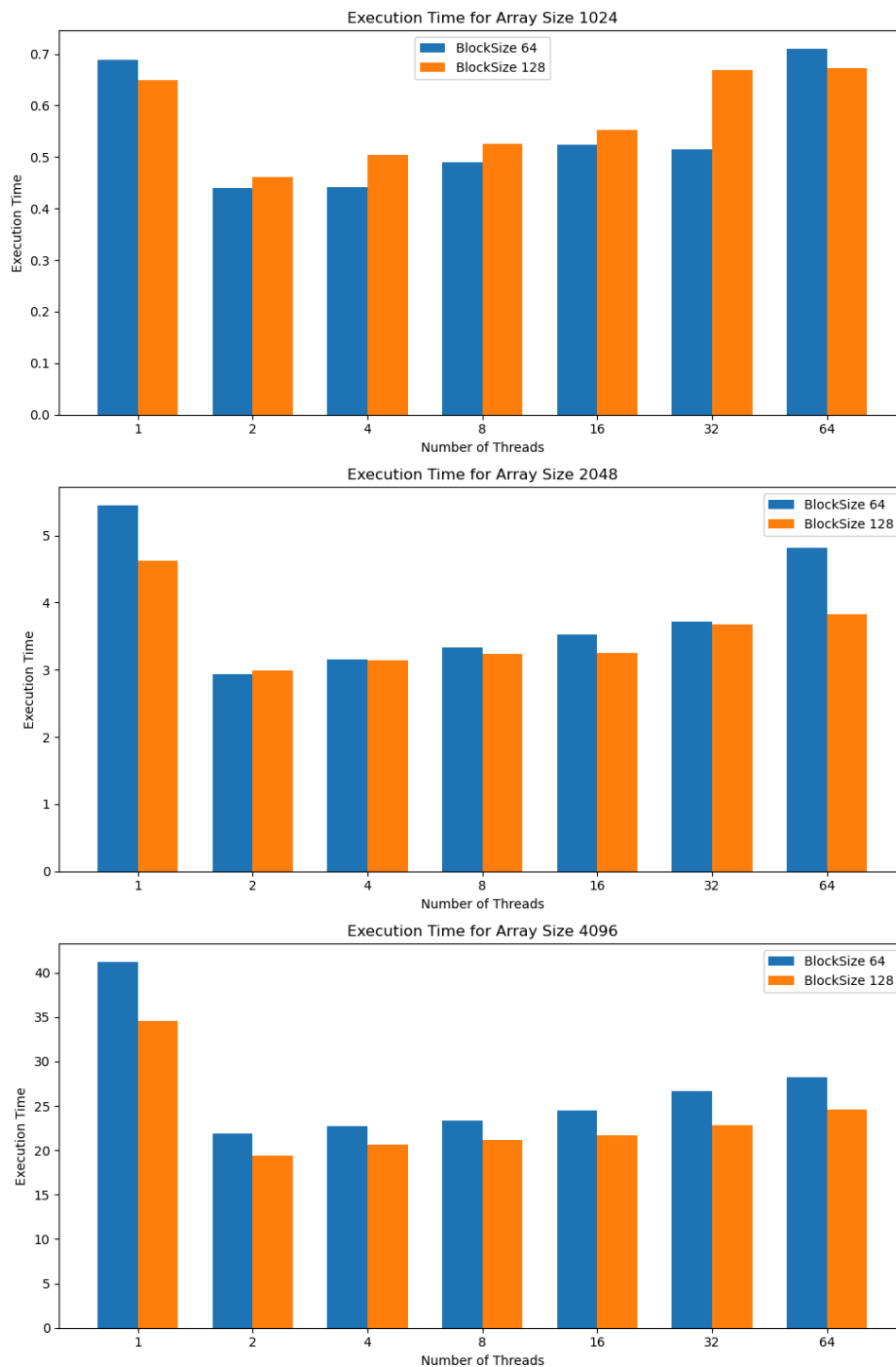
Σε αυτό το ερώτημα παραλληλοποιούμε τον αλγόριθμο fw_sr.c και πραγματοποιούμε μετρήσεις για μεγέθη πινάκων 1024x1024, 2048x2048 και 4096x4096 για threads = {1, 2, 4, 8, 16, 32, 64} στο μηχάνημα sandman.

```
if(myN<=bsize)
    for(k=0; k<myN; k++)
        for(i=0; i<myN; i++)
            for(j=0; j<myN; j++)
                A[arow+i][acol+j]=min(A[arow+i][acol+j], B[brow+i][bcol+k]+C[crow+k][ccol+j]);
else {
    FW_SR(A,arow, acol,B,brow, bcol,C,crow, ccol, myN/2, bsize);
    #pragma omp parallel
    {
        #pragma omp single
        {
            #pragma omp task
                FW_SR(A,arow, acol+myN/2,B,brow, bcol,C,crow, ccol+myN/2, myN/2, bsize);
            #pragma omp task
                FW_SR(A,arow+myN/2, acol,B,brow+myN/2, bcol,C,crow, ccol, myN/2, bsize);
            #pragma omp taskwait
        }
        FW_SR(A,arow+myN/2, acol+myN/2,B,brow+myN/2, bcol,C,crow, ccol+myN/2, myN/2, bsize);
        FW_SR(A,arow+myN/2, acol+myN/2,B,brow+myN/2, bcol+myN/2,C,crow+myN/2, ccol+myN/2, myN/2, bsize);
        #pragma omp parallel
        {
            #pragma omp single
            {
                #pragma omp task
                    FW_SR(A,arow+myN/2, acol,B,brow+myN/2, bcol+myN/2,C,crow+myN/2, ccol, myN/2, bsize);
                #pragma omp task
                    FW_SR(A,arow, acol+myN/2,B,brow, bcol+myN/2,C,crow+myN/2, ccol+myN/2, myN/2, bsize);
                #pragma omp taskwait
            }
        }
        FW_SR(A,arow, acol,B,brow, bcol+myN/2,C,crow+myN/2, ccol, myN/2, bsize);
    }
}
```

Παραλληλοποιούμε τον κώδικα με χρήση task. Βλέπουμε πως λόγω των εξαρτήσεων μπορούμε να παραλληλοποιήσουμε μόνο τις {2,3} και {6,7} αναδρομικές κλήσεις της συνάρτησης FW_SR.

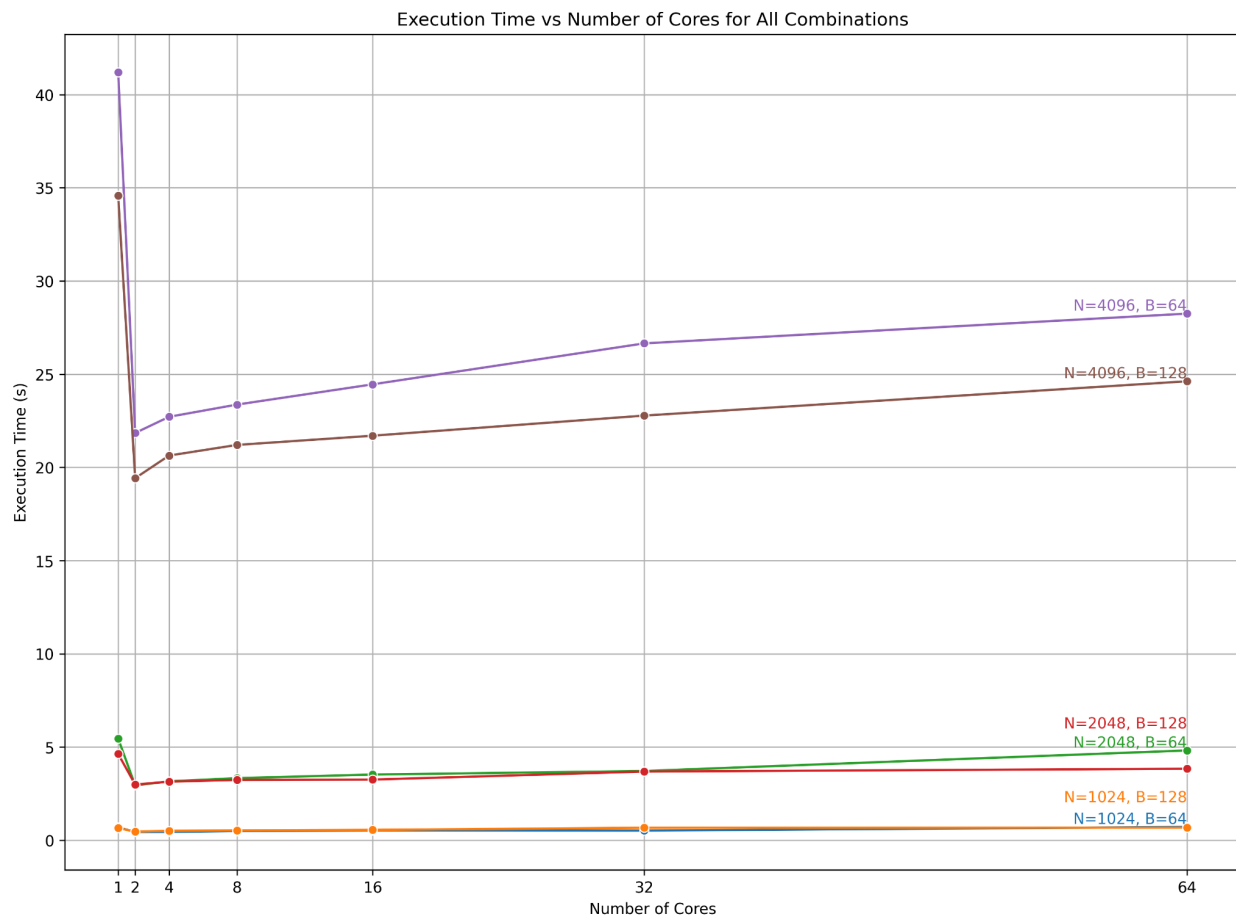
Δοκιμάζουμε και μετράμε χρόνο εκτέλεσης για $N = \{1024, 2048, 4096\}$ και $B = \{64, 128\}$ με Threads= {1,2,4,8,16,32,64}

Παρακάτω λαμβάνουμε τα bar plots για καθε array configuration:



Για 64 threads έχουμε εμφανώς χειρότερη απόδοση ως προς τη κλιμάκωση μέχρι και τα 32 threads, αυτό συμβαίνει διότι λόγω του hyperthreading, το hyperthreading επιχειρεί να έχει δύο pipelines στον ίδιο πυρήνα και να εκμεταλλευτεί το ότι οι δύο αυτές διεργασίες δε θα θελουν ταυτόχρονα τους ίδιους πόρους τη CPU και έτσι μπορούμε να εκμεταλλευτούμε τους νεκρούς αυτούς χρόνους(π.χ cache misses). Στη προκειμένη όμως έχουμε threads στον ίδιο πυρήνα να δουλεύουν στο ίδιο block και επομένως το overhead μειώνει κατα πολύ την απόδοση.

Αναπαριστούμε επίσης σε ένα συνολικό plot τα execution times:

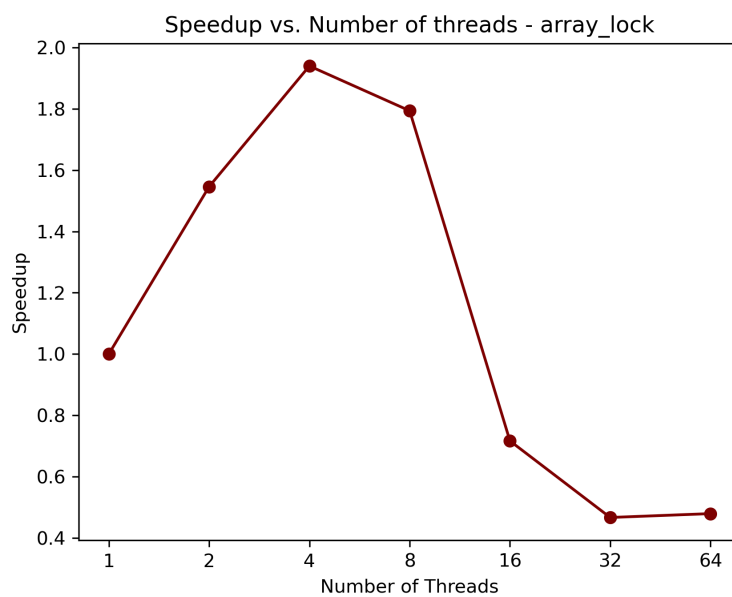
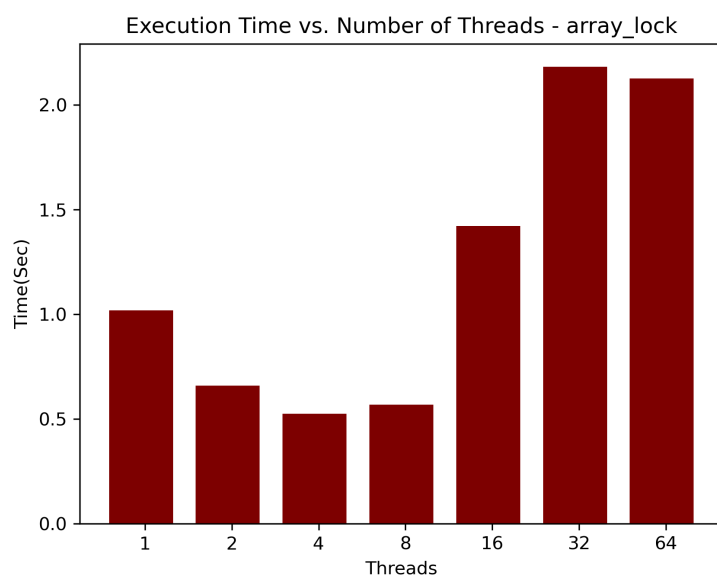


Παρατηρούμε πως βέλτιστη επίδοση έχουμε πάντα στους δύο πυρήνες ανεξαρτήτως των παραμέτρων N και B . Αυτό οφείλεται στο ότι μπορούμε να παραλληλοποιήσουμε μέχρι δυο αναδρομικές κλήσεις την φορά, άρα παραπάνω πυρήνες δεν θα δώσουν περαιτέρω επιτάχυνση καθώς δεν υπάρχουν tasks να τους αξιοποιήσουν.

Επίσης παρατηρούμε πως στα μικρά N το B δεν έχει σημαντικό ρόλο σε αντίθεση με το $N = 4096$ όπου το $B=128$ είναι εμφανώς καλύτερο.

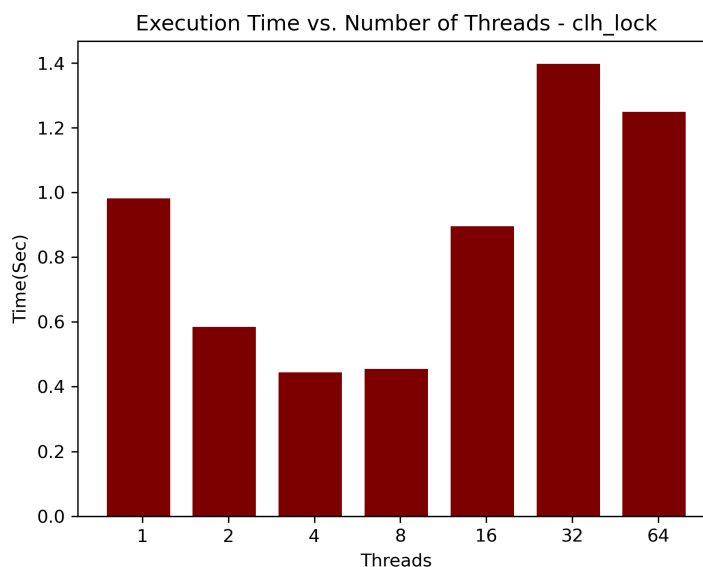
3.1 Παραλληλοποίηση και βελτιστοποίηση του αλγορίθμου K-means - Locks

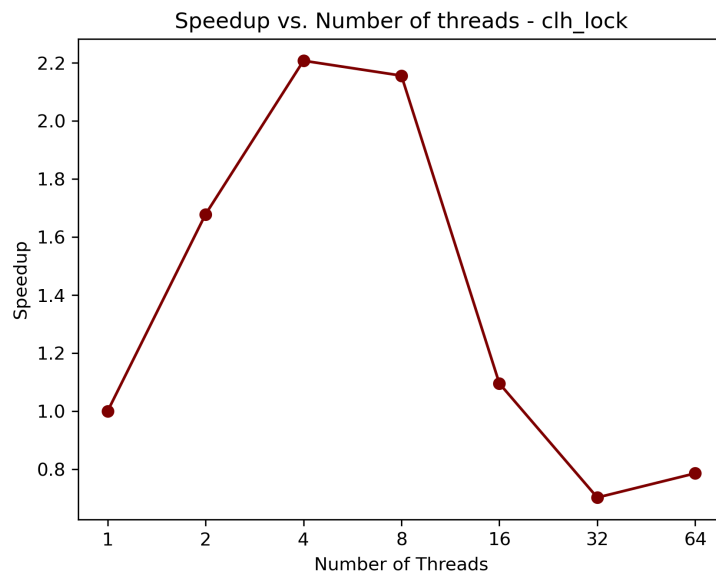
- Array Lock:



Με το array lock δημιουργείται μια λίστα με μέγεθος ίσο με τον αριθμό των threads. Στο τέλος της λίστας υπάρχει ένας counter, ο οποίος αρχικά έχει τιμή μηδέν και ορίζει το τέλος της λίστας. Επιπλέον, το στοιχείο lock[0] αρχικά έχει την τιμή true, ώστε να επιτρέπεται στο πρώτο thread που θα προσπαθήσει να εκτελέσει να τα καταφέρει. Κάθε φορά που ένα thread θέλει να εκτελέσει, λαμβάνει την τιμή του μετρητή και τον αυξάνει κατά ένα. Αναμένει μέχρι το αντίστοιχο στοιχείο του στον πίνακα lock (έστω i) να γίνει lock[i] = true. Όταν αυτό συμβεί, εκτελεί το κρίσιμο τμήμα του κώδικα. Μετά την ολοκλήρωση του κρίσιμου τμήματος, θέτει το lock[i] = false και το lock[(i+1)%size] = true, επιτρέποντας έτσι στην επόμενη διαδικασία να προχωρήσει. Έτσι μειώνεται το contention στο bus γιατί κάθε thread διαβάζει μία ξεχωριστή διεύθυνση μνήμης. Αναμενόμενο λοιπόν είναι και το αποτέλεσμα του χρόνου εκτέλεσης που παρατηρούμε. Απο τα διαγράμματα παρατηρούμε πως το array lock δίνει αθροιστικά λιγότερο χρόνο. Μετά τα 8 threads έχουμε χειρότερη απόδοση καθώς είναι πιο εμφανές το contention για το lock του πίνακα των clusters (οι υπολογισμοί για το κοντινότερο cluster γίνονται μεν γρηγορότερα αλλά αντισταθμίζεται κατα πολύ από το contention για το lock).

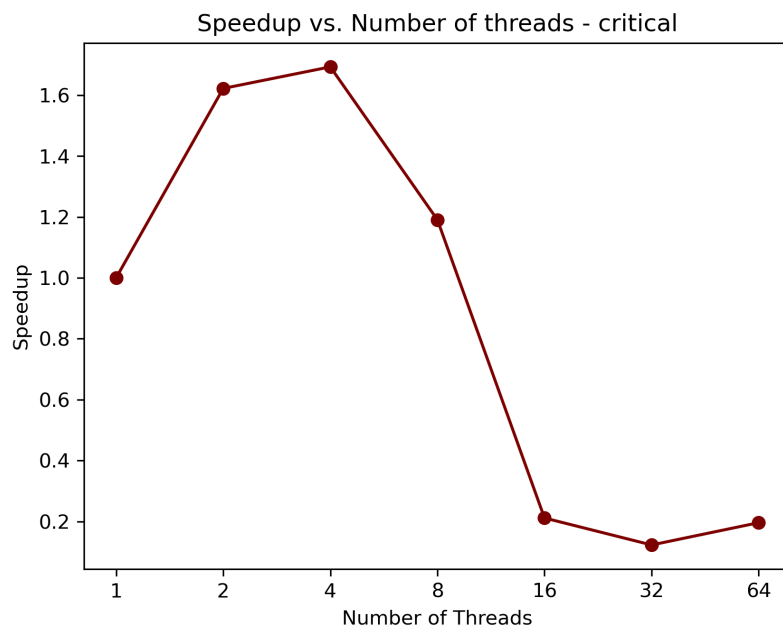
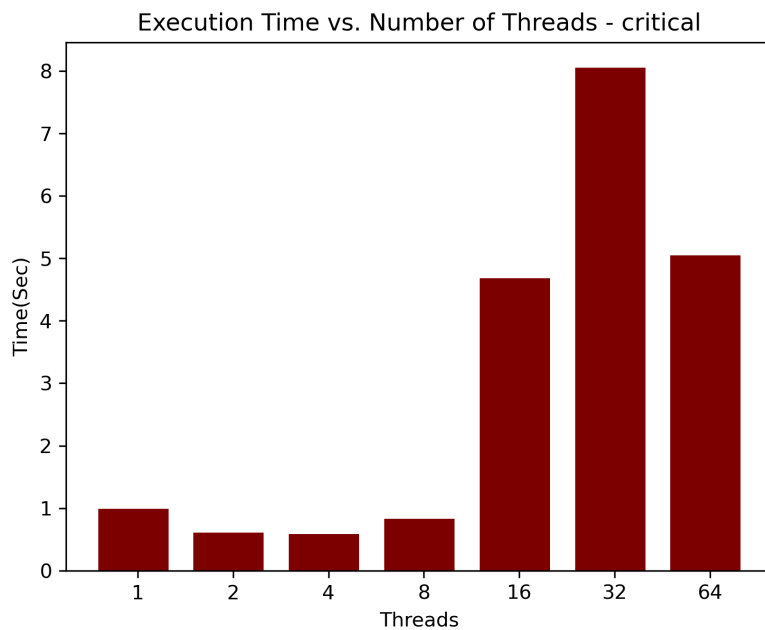
- CLH Lock:





Το clh lock υλοποιεί μια συνδεδεμένη λίστα στην οποία κάθε νήμα που περιμένει να μπει στο κρίσιμο τμήμα δημιουργεί ένα νέο κόμβο στο τέλος της λίστας και θέτει το flag του σε false (δείχνοντας πως θέλει να πάρει το lock). Έτσι κάθε νήμα κάνει spin το flag του προγόνου του. Το clh lock είναι αποδοτικότερο κλείδωμα ιδιαίτερα για critical sections που δε διαρκούν μεγάλο χρονικό διάστημα διότι είναι τύπου spin lock και σε αυτές τις περιπτώσεις είναι αποδοτικότερο να κάνει spin σε μία μεταβλητή απο το να βάζουμε σε sleep τα νήματα και να ξανα ξυπνάμε. Το clh lock μας δίνει τους καλύτερους χρόνους εκτέλεσης, λόγω του spin σε δικιά του διεύθυνση για κάθε νήμα και της μείωσης των Cache Invalidations καθώς γίνεται invalidate μόνο η cache line του επόμενου στη σειράς νήματος πετυχαίνουμε τους καλύτερους χρόνους.

- Critical:

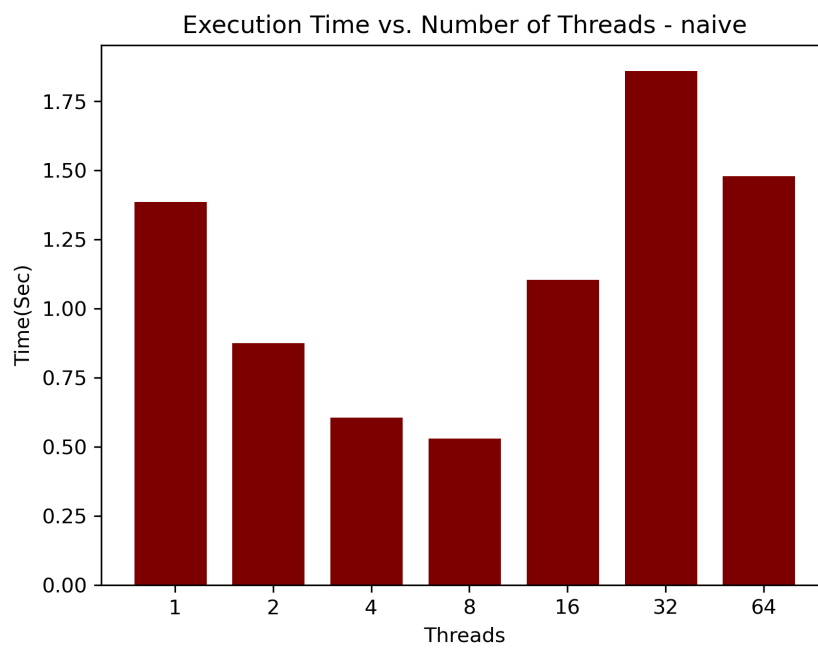


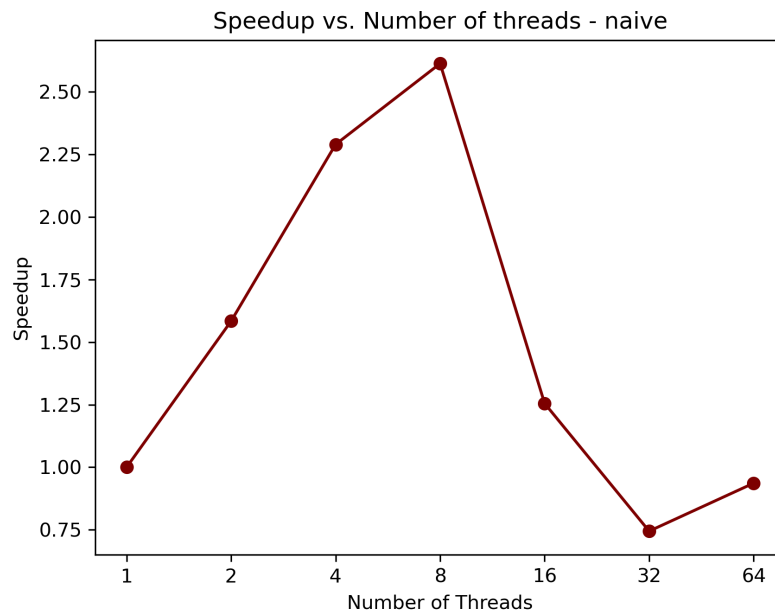
Στην critical υλοποίηση ο συγχρονισμός γίνεται απο το API της OpenMP, συγκεκριμένα παρεμβάλει πριν και μετά από το critical section την εξής “εντολή”:

```
#pragma omp critical
```

```
14:     #pragma omp critical
15:     {
16:         newClusterSize[index]++;
17:         for (j=0; j<numCoords; j++){
18:             newClusters[index*numCoords + j] += objects[i*numCoords + j];
19:         }
20:     }
21: }
```

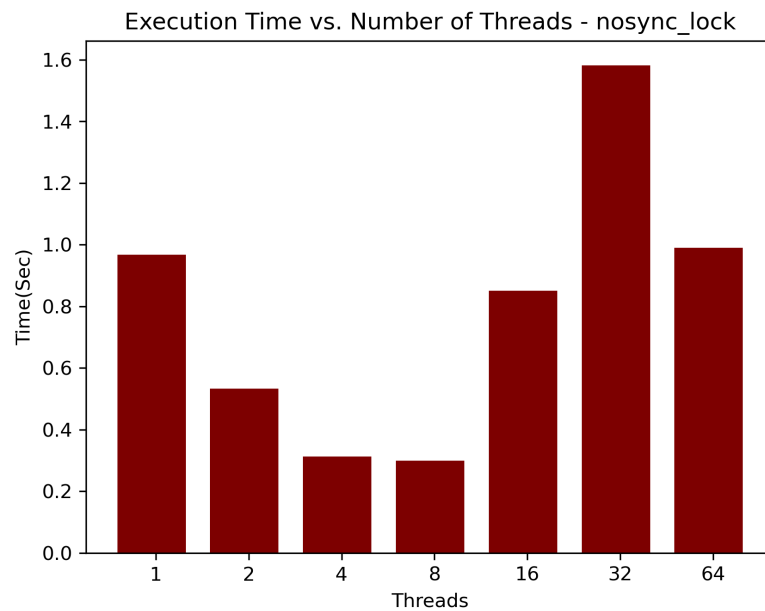
- Naive:

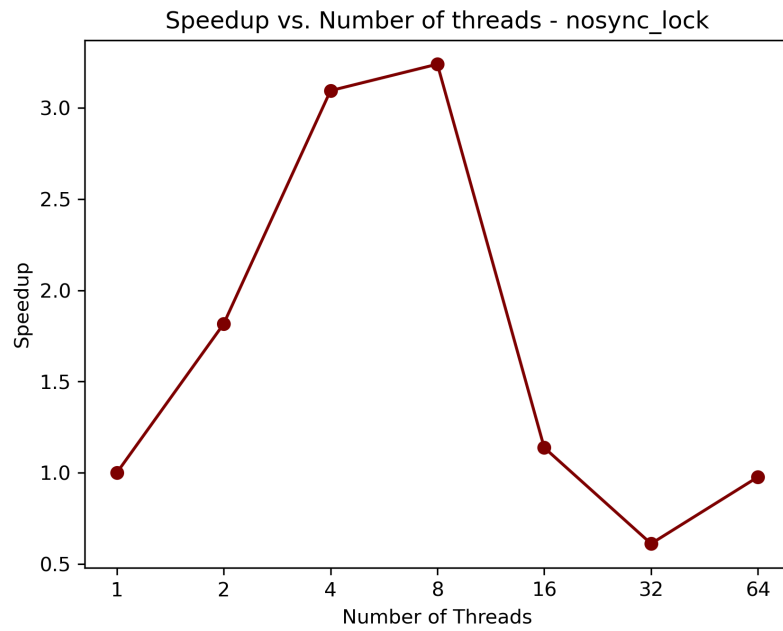




Στη Naive υλοποίηση χρησιμοποιούμε `#pragma omp atomic`, πριν τη πρόσβαση στο `newClusters` και `newClusterSize` ώστε να εξασφαλίσουμε atomic access σε αυτά.

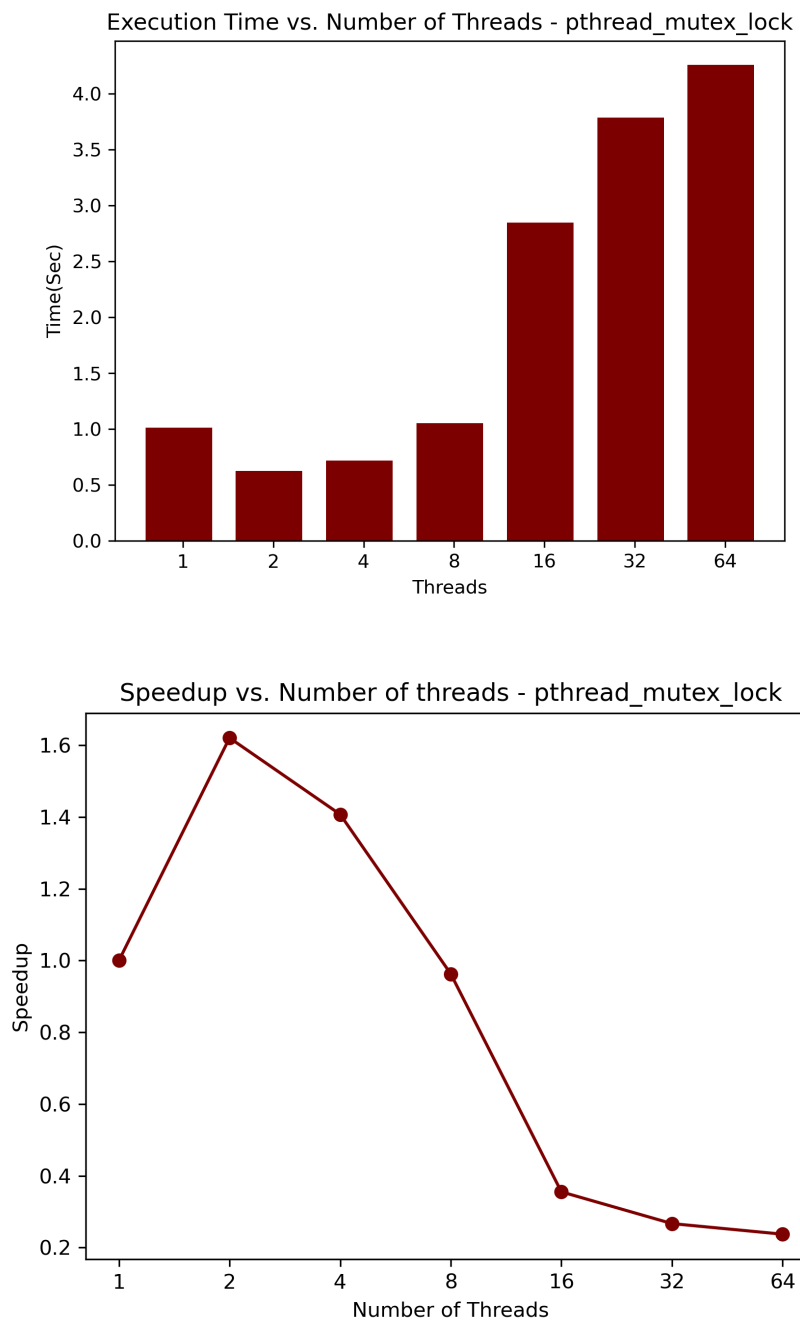
- NoSync Lock:





Το nosync lock όπως είναι αναμενόμενο έχει το λιγότερο συνολικό χρόνο για κάθε configuration νημάτων, δε φροντίζει για ατομικό access στα newClusters και newClusterSize και επομένως το clustering που κάνει είναι λάθος.

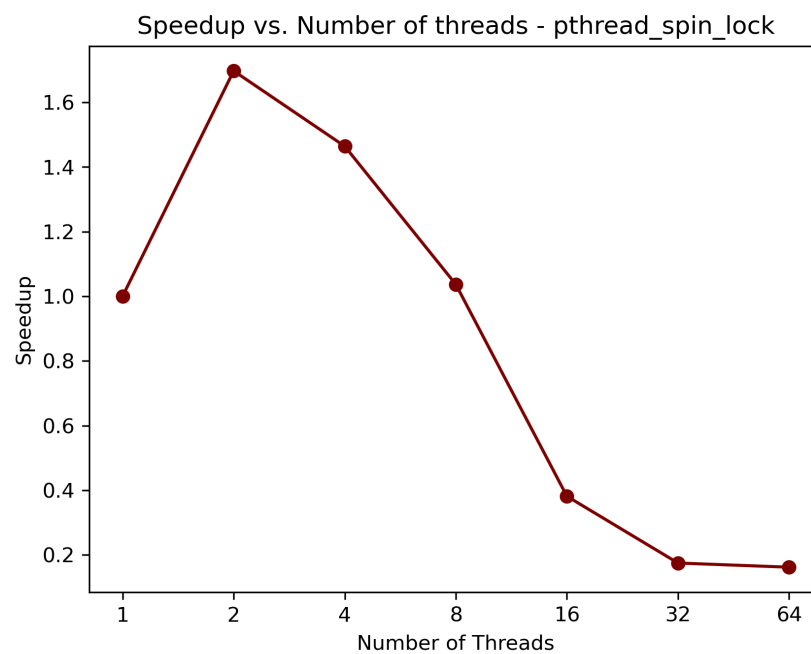
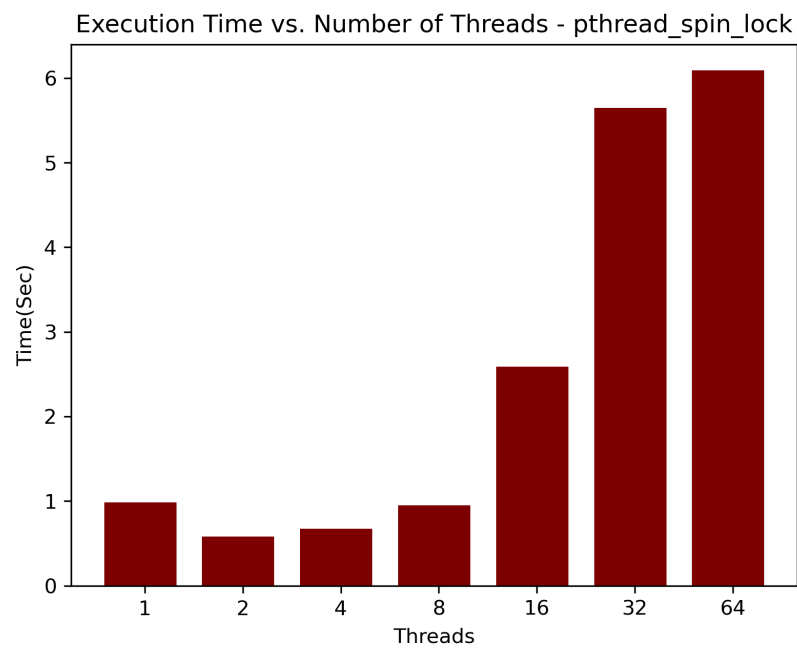
- Pthread Mutex Lock:



Στην υλοποίηση αυτή κάθε νήμα περιμένει να πάρει το lock κάνοντας κλήση στη συνάρτηση `acquire_lock()`, αν το mutex δεν είναι ελεύθερο το νήμα περιμένει (sleep) μέχρι να είναι ελεύθερο. Ξανα όπως και στις άλλες υλοποιήσεις μετά τα 8 threads έχουμε χειρότερη απόδοση καθώς είναι πιο εμφανές το contention για το lock του

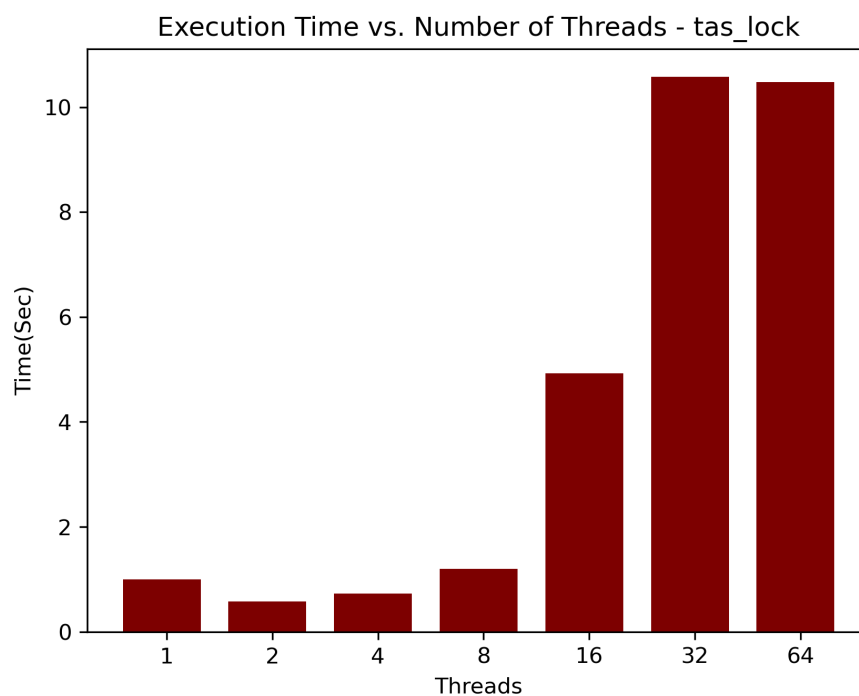
πίνακα των clusters (οι υπολογισμοί για το κοντινότερο cluster γίνονται μεν γρηγορότερα αλλά αντισταθμίζεται κατα πολύ απο το contention για το lock).

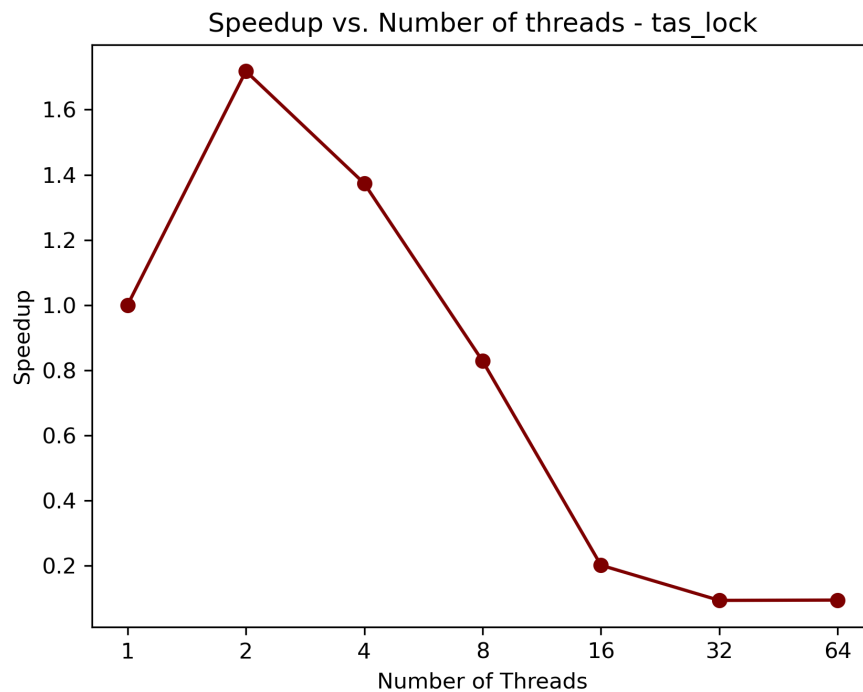
- Pthread Spin Lock:



Σε αυτή την υλοποίηση το thread προσπαθεί να πάρει το lock κάνοντας συνέχεια spin (τσεκάρει συνεχώς τη διαθεσιμότητα του lock) μέχρι να καταφέρει να το πάρει. Δεν εξασφαλίζεται κάποια σειριοποίηση αν και στη προκειμένη δε μας ενδιαφέρει. Ξανα όπως και στις άλλες υλοποιήσεις μετά τα 8 threads έχουμε χειρότερη απόδοση καθώς είναι πιο εμφανές το contention για το lock του πίνακα των clusters (οι υπολογισμοί για το κοντινότερο cluster γίνονται μεν γρηγορότερα αλλά αντισταθμίζεται κατα πολύ απο το contention για το lock).

- TAS Lock





Η υλοποίηση αυτή βασίζεται στη λογική Test and Set, το κάθε νήμα με χρήση ατομικής εντολής `__sync_lock_test_and_set` γράφει locked στο state του lock και επιστρέφει πίσω την παλιά τιμή του state, αν δεν ήταν unlocked συνεχίζει το while loop.

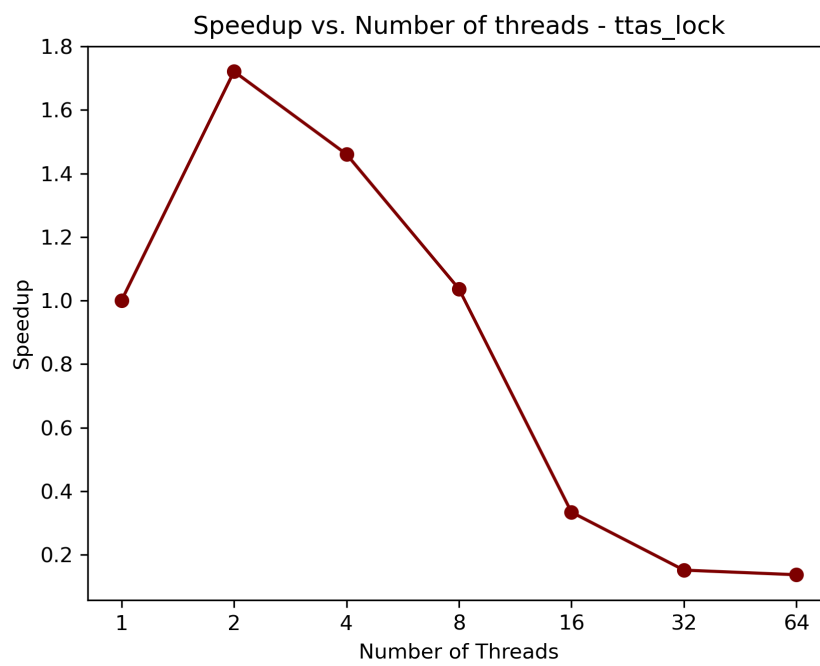
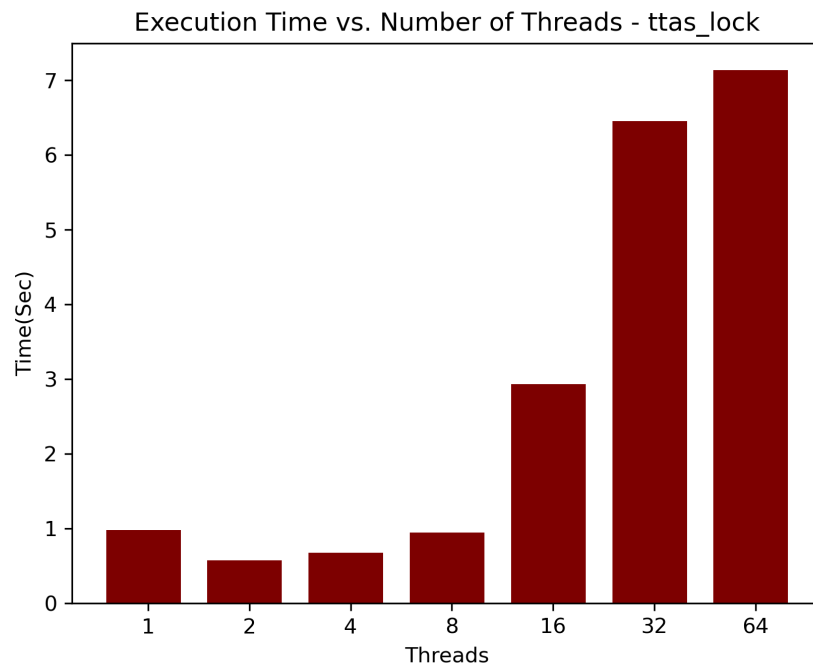
The `while` loop uses the `__sync_lock_test_and_set` function to attempt to acquire the lock. This function is a built-in function provided by GCC for atomic operations. It takes two arguments: a pointer to a memory location and a new value. The function atomically sets the value at the given memory location to the new value and returns the old value.

```
void lock_acquire(lock_t *lock)
{
    lock_t *l = lock;

    while (__sync_lock_test_and_set(&l->state, LOCKED) == LOCKED)
        /* do nothing */ ;
}
```

Ξανα όπως και στις άλλες υλοποιήσεις μετά τα 8 threads έχουμε χειρότερη απόδοση καθώς είναι πιο εμφανές το contention για το lock του πίνακα των clusters (οι υπολογισμοί για το κοντινότερο cluster γίνονται μεν γρηγορότερα αλλά αντισταθμίζεται κατά πολύ από το contention για το lock).

- TTAS Lock:



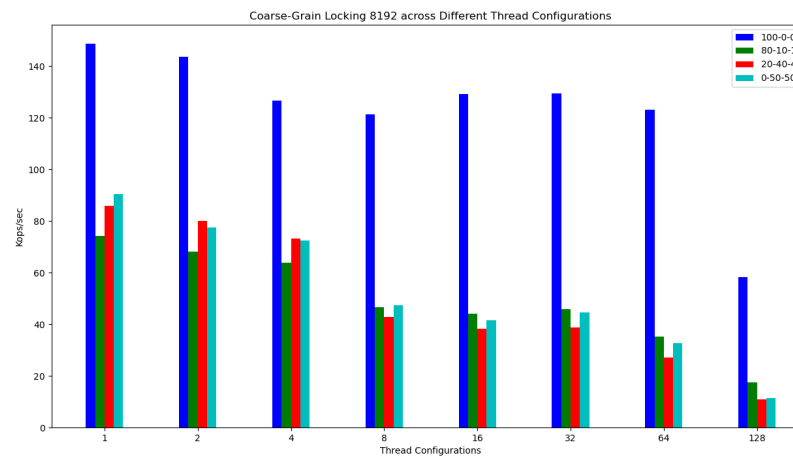
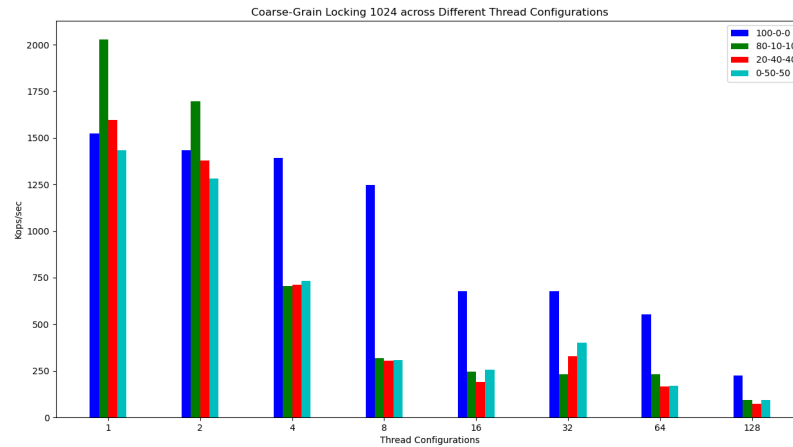
Σε αντίθεση με το `tas_lock`, το `ttas_lock` προσπαθεί να αποκτήσει το κλείδωμα μόνο όταν διαπιστώσει ότι το κλείδωμα είναι ελεύθερο. Αυτό σημαίνει ότι η ατομική εντολή που γράφει στην κοινόχρηστη μνήμη εκτελείται λιγότερες φορές, άρα έχουμε και λιγότερη κίνηση στο bus. Επομένως, αναμένουμε να δούμε βελτίωση της απόδοσης σε σχέση με πριν. Κάτι που παρατηρούμε καθώς σε όλα τα threads βλέπουμε μειωμένο χρόνο εκτέλεσης.

```
void lock_acquire(lock_t *lock)
{
    lock_t *l = lock;

    do {
        while (l->state == LOCKED)
            /* do nothing */ ;
    } while (__sync_lock_test_and_set(&l->state, LOCKED) == LOCKED);
}
```

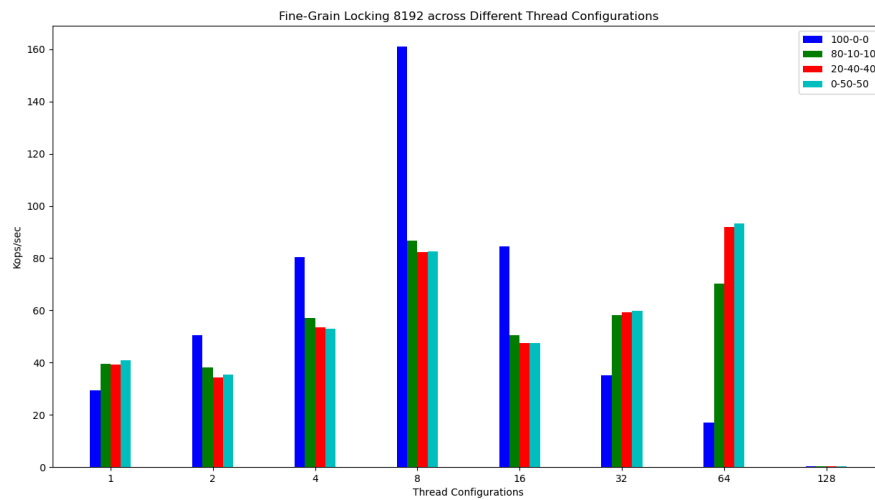
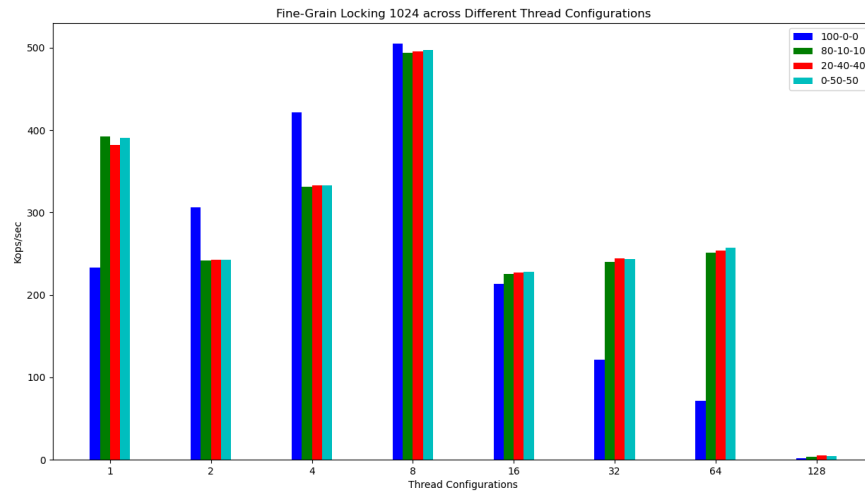
2.3 Ταυτόχρονες Δομές Δεδομένων

- **Coarse-Grain Locking**



Στο coarse-grain locking για κάθε operation στη λίστα κλειδώνεται όλη η λίστα. Αυτό οδηγεί ,όπως είναι αναμενόμενο, στο να μειώνονται τα operations ανα second όσο περισσότερα threads εμπλέκουμε, τόσο στο μέγεθος 1024 όσο και στο 8192.

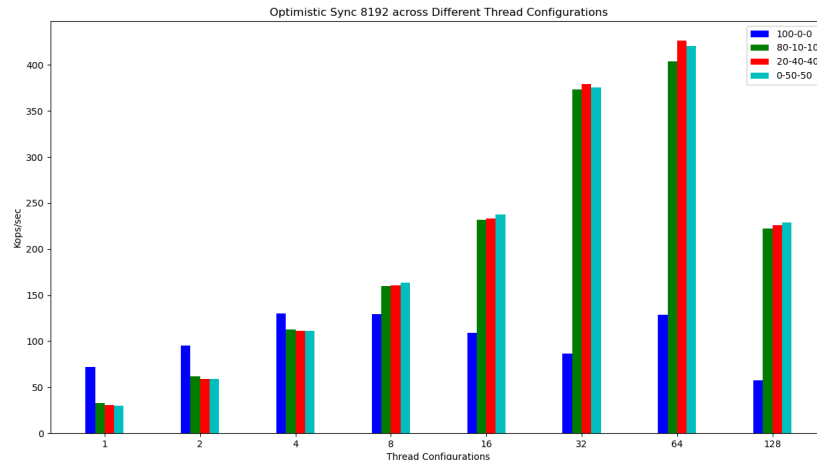
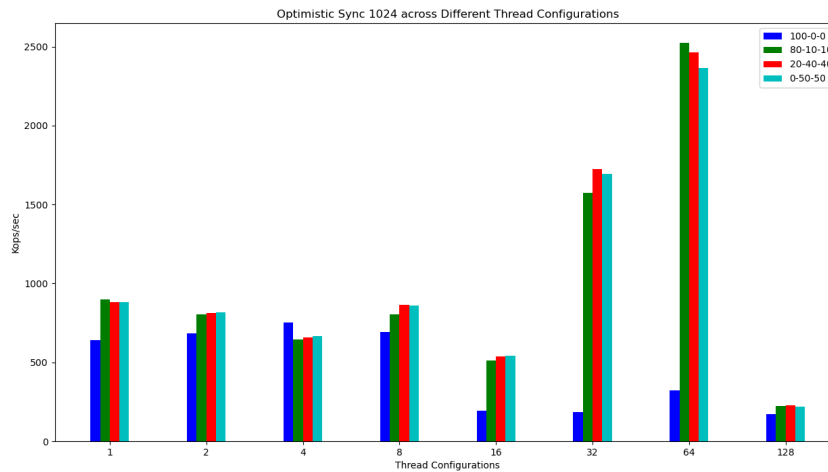
- **Fine-Grain Locking**



Στο fine grain locking δε κλειδώνει όλη τη λίστα καθε thread που θέλει να πραγματοποιήσει ένα operation, αλλα κλειδώνει σε επίπεδο node. Ένα νήμα πρέπει να πάρει το lock του πριν μπορέσει να το διαβάσει ή να εκτελέσει μια πράξη όπως εισαγωγή ή διαγραφή. Όπως είναι αναμενόμενο η απόδοση είναι ανοδική με αύξηση των threads. Ωστόσο χειροτερεύει όταν έχουμε μεγάλο ποσοστό αναζητήσεων, λόγω του hand-over-hand locking τα thread που θα

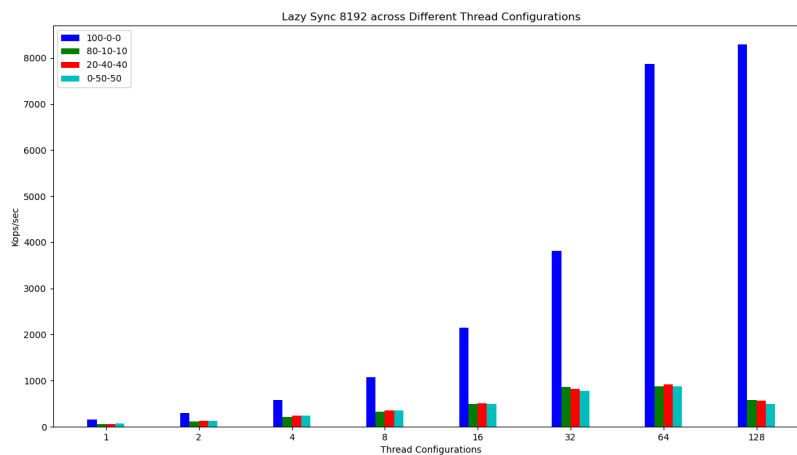
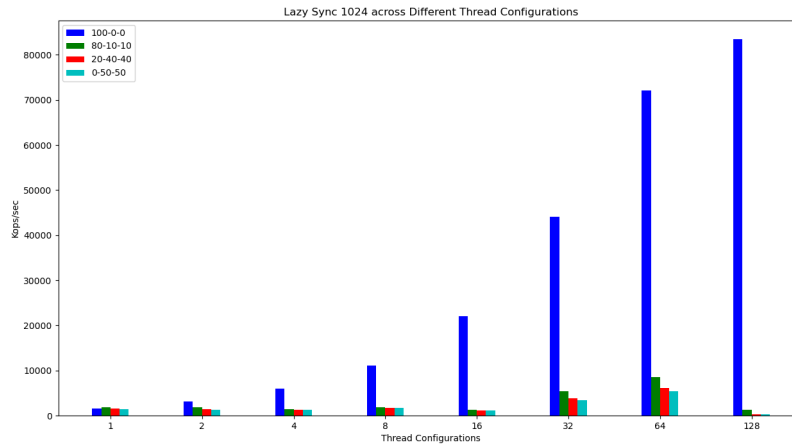
θέλουν να διαβάσουν πέρα από το locked node απο ένα άλλο thread θα δημιουργούν “κίνηση” πίσω από αυτό, με αποτέλεσμα να σπαταλούν ώρα σε αναμονή παρά στη πράξη τους.

- **Optimistic Synchronization**



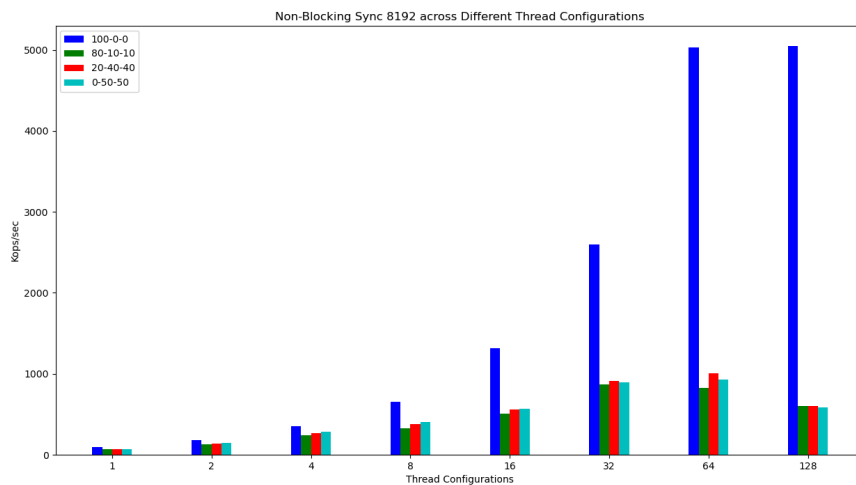
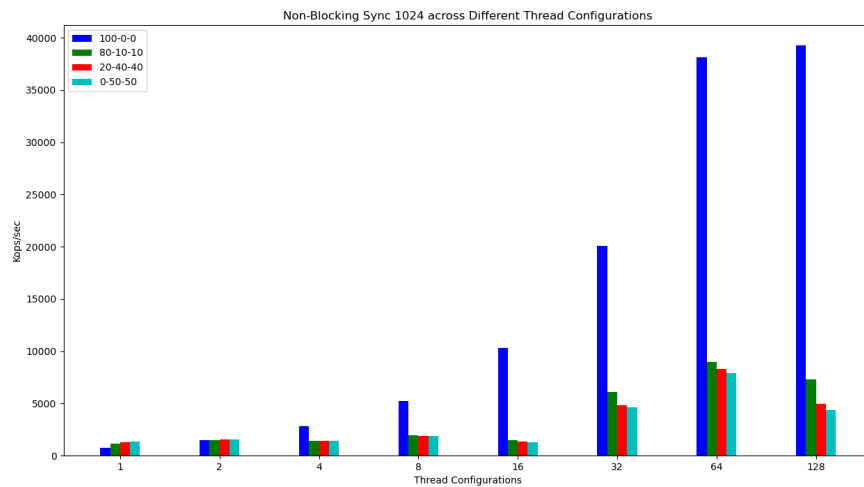
Το optimistic synchronization βασίζεται στην ιδέα οτι η σύγκρουση μεταξύ νημάτων δεν είναι συχνή, με αυτό το σκεπτικό κάθε νήμα αρχικά δουλεύει χωρίς locks και εκτελεί τα operations και μόνο στο τέλος κανει validate οτι δεν υπήρξε σύγκρουση με operation άλλου thread. Η διαφορά στην απόδοση συγκριτικά με άλλα locking schemas είναι πιο εμφανής στα configurations που περιλαμβάνουν εισαγωγές/διαγραφές. Ένα σημαντικό μειονέκτημα είναι ότι είναι πιθανό το starvation καθώς σε συστήματα που έχουν μεγάλη κίνηση είναι πιθανό τα νήματα να αποτυγχάνουν να κάνουν validate και να ξαναπροσπαθούν ξανα και ξανα.

- **Lazy Synchronization**



Στο Lazy synchronization προσθέτουμε στη δομή μία boolean μεταβλητή που δείχνει αν ο κόμβος βρίσκεται στη λίστα ή έχει διαγραφεί από αυτή. Η κύρια διαφορά σε σχέση με το optimistic synchronization είναι ότι η μέθοδος contains δε χρειάζεται να διατρέξει την λίστα από την αρχή σε περίπτωση που το validate αποτύχει. Γενικά παρατηρούμε πως πετυχαίνουμε “θετικό” scaling καθώς ανεβαίνουμε τα thread configurations.

- **Non-Blocking Synchronization**

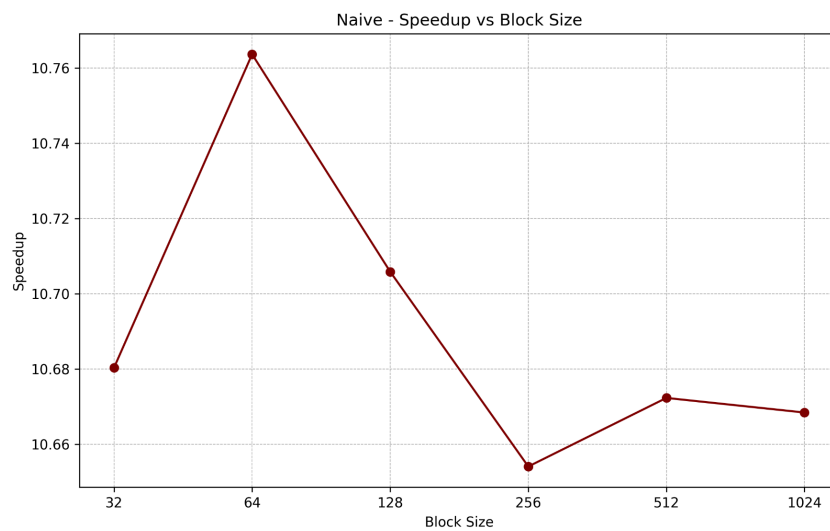
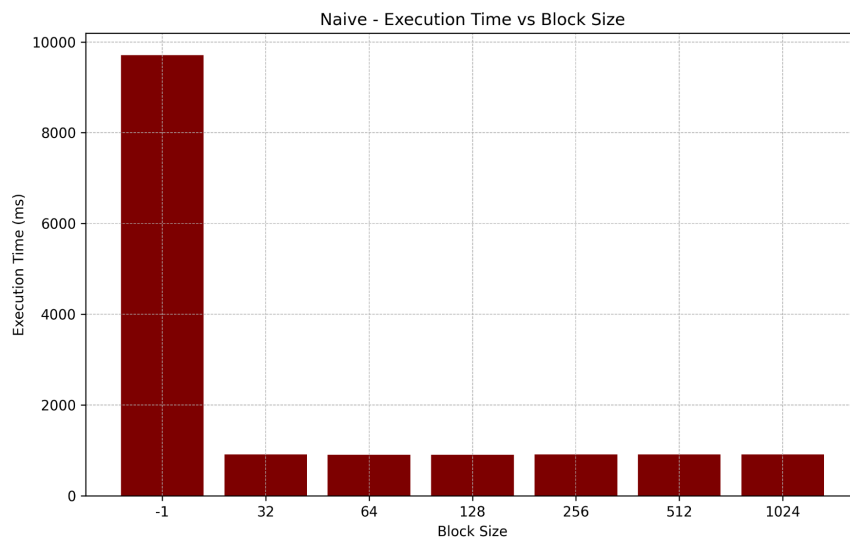


Το non-blocking synchronization αποτελεί τη λύση στην χρήση lock της lazy synchronization στα insert και delete tasks. Το overhead των locks απαλείφεται και αντικαθιστάται με reruns στην περίπτωση που δεν ισχύει η τωρινή κατάσταση κάποιου εκ των εξεταζόμενων κόμβων. Το μεγαλύτερο μειονέκτημα αυτής της στρατηγικής είναι η επιβάρυνση της find. Αυτή, κατά τη διάσχιση της λίστας, κάνει έναν έλεγχο σε κάθε κόμβο, ώστε να δει αν είναι marked. Αυτό επιτυγχάνεται συγχωνεύοντας την boolean που αναφέραμε στην Lazy υλοποίηση με το πεδίο next του κόμβου, το οποίο δείχνει στο επόμενο στοιχείο της λίστας. Έτσι, η boolean αυτή λειτουργεί τώρα ως flag που επιτρέπει την ενημέρωση του πεδίου next του κόμβου, εφόσον αυτός υπάρχει λογικά. Έχουμε πάλι “θετικό” scaling μέχρι και τα 64 threads.

4. Παραλληλοποίηση και βελτιστοποίηση αλγορίθμων σε επεξεργαστές γραφικών

Μέχρι και *bottleneck analysis* έχουμε το default Config:{256,2,16,10}

Naive Implementation:



Για τη παράλληλη υλοποίηση αναθέτουμε στη GPU τον υπολογισμό των αποστάσεων των objects απο τα κεντρα των clusters, αναθέτουμε σε κάθε thread ένα object. Παρατηρούμε στο διαγραμμα του χρόνου εκτέλεσης τεράστια διαφορά μεταξύ σειριακής υλοποίησης (ορισμένη αυθαίρετα με block_size = -1) και παράλληλης υλοποίησης. Η παράλληλη υλοποίηση είναι μια τάξη (~10) μεγέθους μικρότερη σε χρόνο.

Το block size δε φαίνεται να παίζει ιδιαίτερο ρόλο για αυτήν την έκδοση. Για να έχουμε καλύτερη εικόνα για το πως επηρεάζει το block size οφείλουμε να γνωρίζουμε παραπάνω πληροφορίες για το hardware στο οποίο τρέχει η εφαρμογή.

Με το παρακάτω C++ αρχείο και με τη βοήθεια του CUDA API που παρέχει τη συνάρτηση cudaGetDeviceProperties() λαμβάνουμε πληροφορίες για πληροφορίες όπως maxThreadsPerBlock ώστε να σχηματίσουμε πληρέστερη εικόνα:

```
#include <iostream>
using namespace std;

int main()
{

    cout << "======" << endl << endl;
    cout << "CUDA version:   v" << CUDART_VERSION << endl;

    int devCount;
    const int kb = 1024;
    const int mb = kb * kb;
    cudaGetDeviceCount(&devCount);
    cout << "CUDA Devices: " << endl << endl;

    for(int i = 0; i < devCount; ++i)
    {
        cudaDeviceProp props;
        cudaGetDeviceProperties(&props, i);
        cout << i << ": " << props.name << ": " << props.major << "." <<
props.minor << endl;
        cout << "  Global memory:   " << props.totalGlobalMem / mb << "mb" <<
endl;
        cout << "  Shared memory:   " << props.sharedMemPerBlock / kb << "kb" <<
endl;
        cout << "  Constant memory: " << props.totalConstMem / kb << "kb" <<
endl;
        cout << "  Block registers: " << props.regsPerBlock << endl << endl;
    }
}
```

```

        cout << "  Max threads per Multiprocessor: " <<
props.maxThreadsPerMultiProcessor << endl;
        cout << "  Max threads per Block: " << props.maxThreadsPerBlock << endl;
        cout << "  Number of Multiprocessors: " << props.multiProcessorCount <<
endl << endl;

        cout << "  Warp size:          " << props.warpSize << endl;
        cout << "  Threads per block: " << props.maxThreadsPerBlock << endl;
        cout << "  Max block dimensions: [ " << props.maxThreadsDim[0] << ", "
<< props.maxThreadsDim[1] <<
", " << props.maxThreadsDim[2] << " ]" << endl;
        cout << "  Max grid dimensions: [ " << props.maxGridSize[0] << ", " <<
props.maxGridSize[1] << ", "
<< props.maxGridSize[2] << " ]" << endl;
        cout << endl;
    }
}

```

Χρησιμοποιούμε ως baseline μια Tesla V100-SXM2-32GB GPU για όλες τις μετρήσεις μας:

```

1: Tesla V100-SXM2-32GB: 7.0
Global memory: 32501mb
Shared memory: 48kb
Constant memory: 64kb
Block registers: 65536

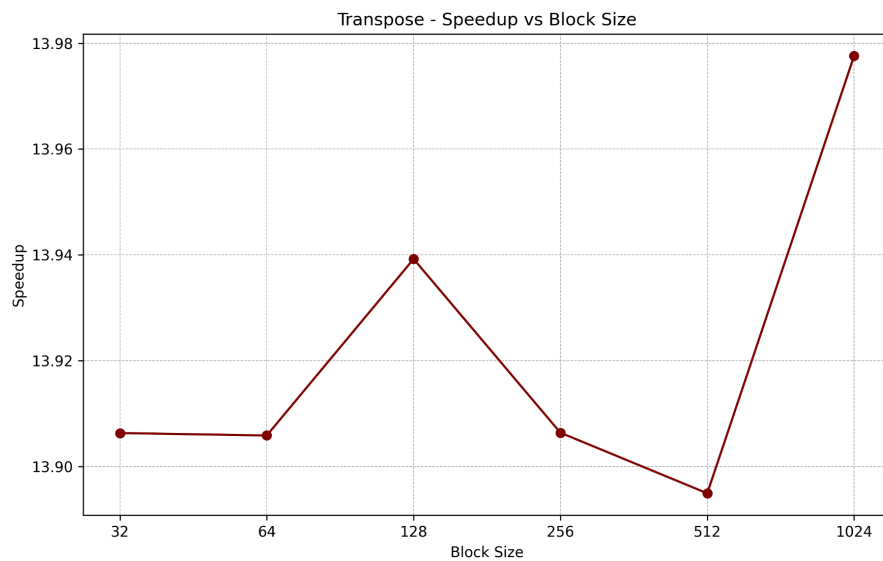
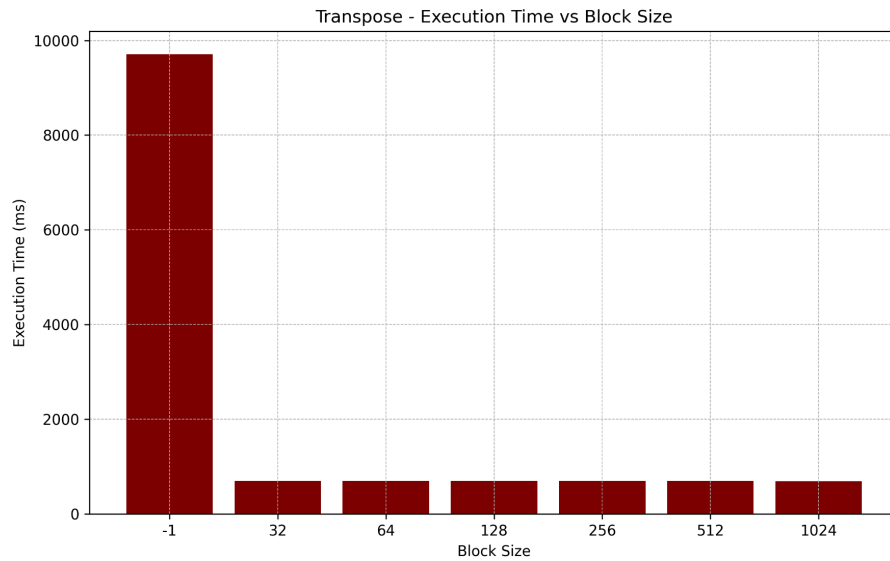
Max threads per Multiprocessor: 2048
Max threads per Block: 1024
Number of Multiprocessors: 80

Warp size: 32
Threads per block: 1024
Max block dimensions: [ 1024, 1024, 64 ]
Max grid dimensions: [ 2147483647, 65535, 65535 ]

```

Στη Naive υλοποίηση έχουμε peak performance για block size 128, χωρίς όμως να διαφέρει ιδιαίτερα το speedup συγκριτικά με τα υπόλοιπα block_sizes > 32. Επομένως φαίνεται να μην έχουμε αξιοσημείωτο hit στην απόδοση παίζοντας με το block size.

Transpose:



Στη Transpose version αλλάζει μόνο η δομή των clusters πινάκων:

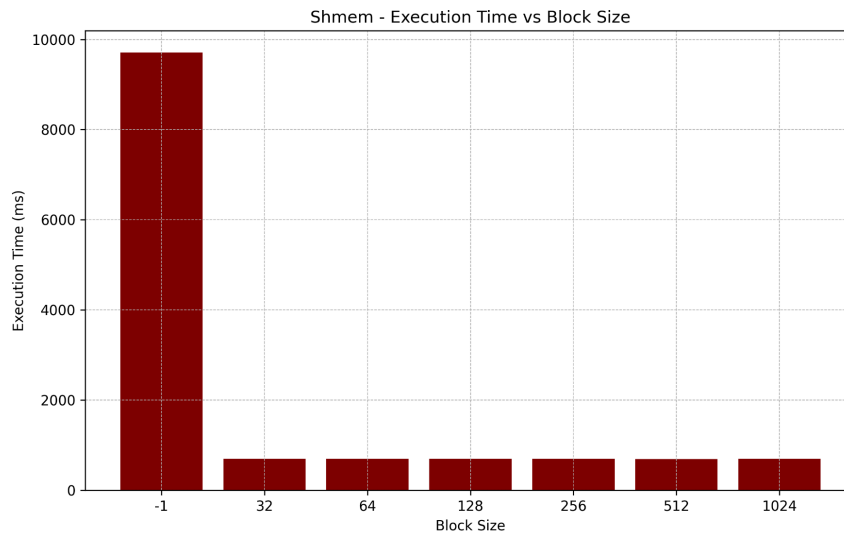
```
double *clusters, // [numCoords][numClusters]
```

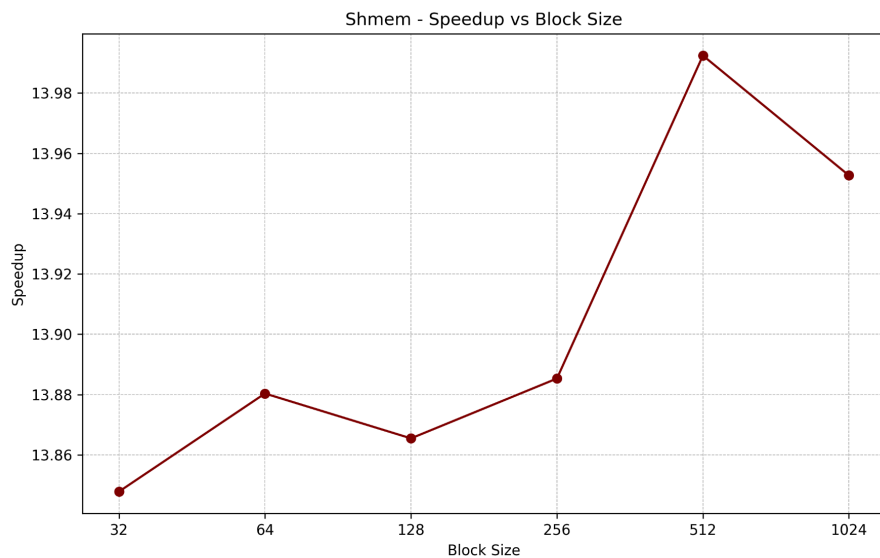
και πλέον έχουμε την ανάστροφη μορφή της Naive:

```
double *clusters, // [numClusters][numCoords]
```

Αυτή αλλαγή ωφελεί το access στη RAM όταν επιθυμούμε να φορτώσουμε και να μεταφέρουμε δεδομένα στη GPU. Τα coordinates αυτά, χάρη στην transpose συνάρτηση, βρίσκονται σε γειτονικές θέσεις μνήμης (στη naive βρίσκονται σε γειτονικές θέσεις τα διαφορετικά coordinates του ίδιου object) και μπορεί να τα πάρει μαζικά και να τα διαμοιράσει στα threads που είναι υπεύθυνα για τον υπολογισμό. Έτσι μειώνουμε τη καθυστέρηση λόγω accesses καθώς μπορεί η GPU να μαζέψει συνολικά τα objects με ένα access. Ός προς τη αλλαγή συμπεριφοράς με block size δεν αλλάζει κάτι.

Shared:





Στη shared υλοποίηση αποθηκεύουμε τα clusters στην κοινή μνήμη της GPU. Με αυτόν τον τρόπο αποφεύγονται τα accesses στην κύρια μνήμη για τον υπολογισμό της απόστασης του κάθε object από όλα τα clusters, τα οποία στοιχίζουν χρόνο λόγω του latency. Δεσμεύουμε λοιπόν μνήμη μεγέθους `numCoords*numClusters*sizeof(double)`

Παρατηρούμε γρηγορότερη ή και ίση σε κάποιες φάσεις) εκτέλεση από τις Naive και Transpose υλοποιήσεις, αναμενόμενο λόγω της γρήγορης shared μνήμης χωρίς την ανάγκη για πολλά accesses στη κύρια μνήμη.

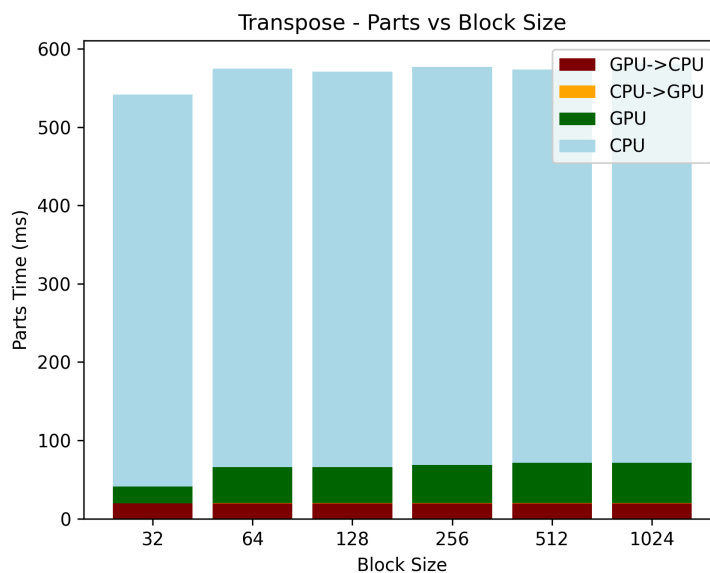
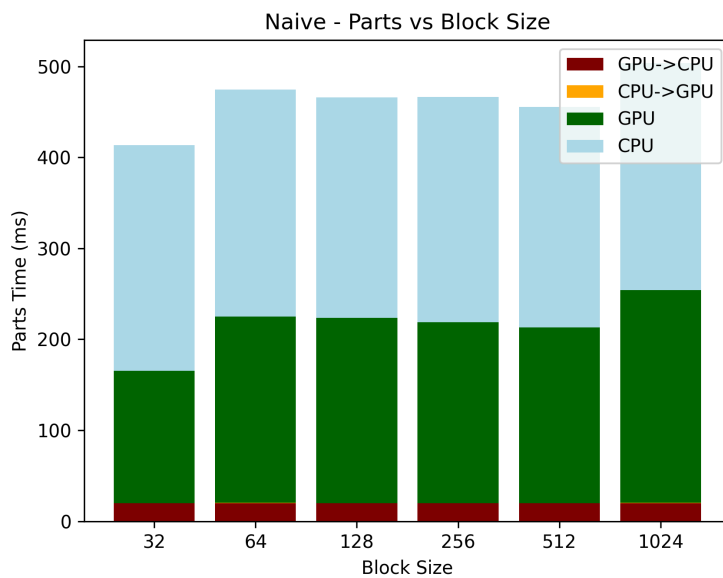
Γενικές Παρατηρήσεις:

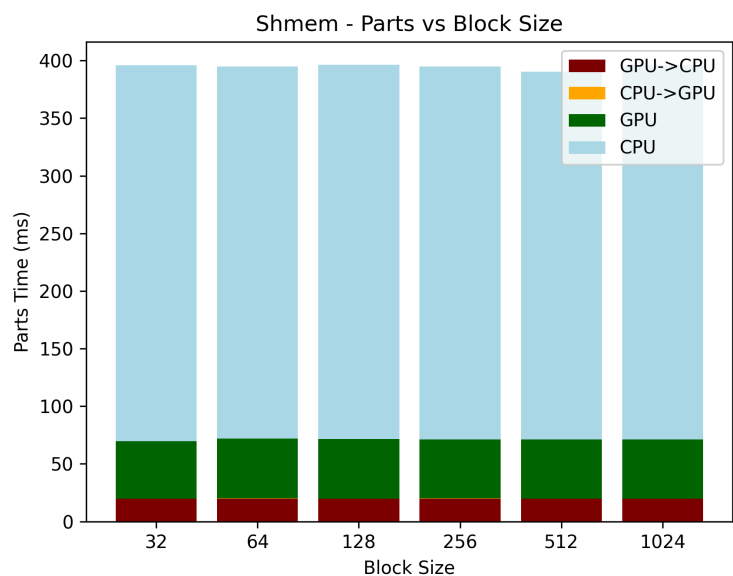
Το speedup αυξάνεται σημαντικά σε σχέση με την naive και την transpose εκδοση. Αυτό συμβαίνει διότι κάθε thread εκτελεί προσβάσεις σε όλα τα clusters, επομένως έχοντας τα στην κοινή μνήμη στην οποία τα φέρνουμε μία φορά όλα μαζί χρησιμοποιώντας πολλά threads γλιτώνουμε πολλά χρονοβόρα accesses.

Σχετικά με *block size* θα περιμέναμε η βέλτιστη επιτάχυνση να επιτυγχάνεται ούτε σε πολύ μεγάλο ούτε σε πολύ μικρό *block size*, στο πρώτο μπορεί να οδηγούμαστε σε *memory bank conflicts* καθώς όταν πολλά *threads* δουλεύουν στο ίδιο *memory bank* οδηγούμαστε σε σειριοποίηση κάποιων από αυτά, στο δεύτερο θα είχαμε πάρα πολλές αντιγραφές αυξάνοντας το *latency*. Αν κοιτάξουμε τα διαγράμματα *speedup* για όλες τις περιπτώσεις φαίνεται το βέλτιστο *block size* να είναι το 128.

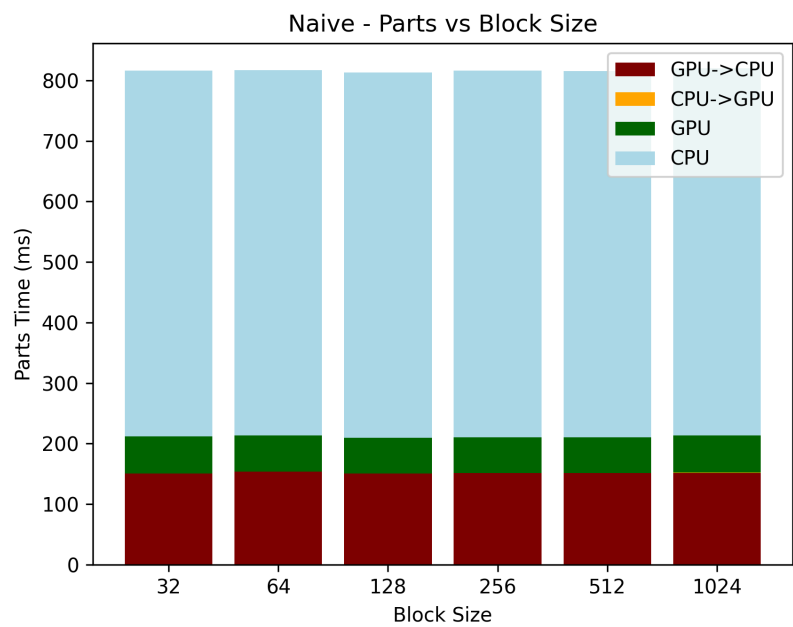
Bottleneck Analysis

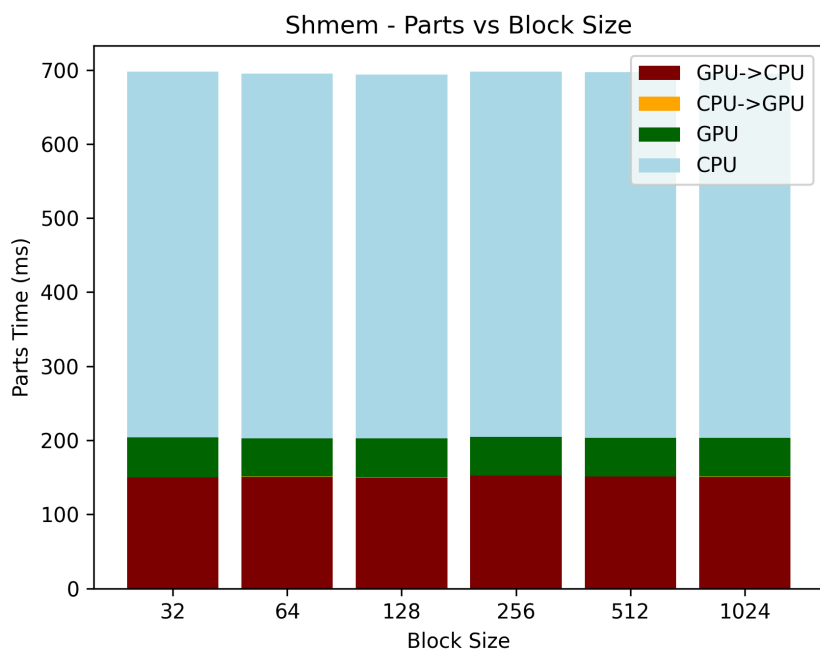
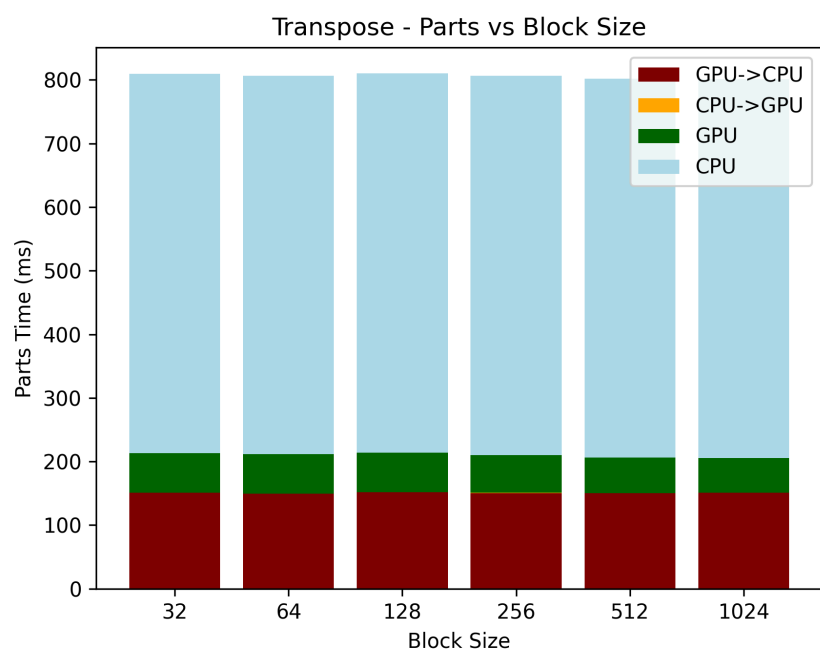
Config {256,16,16,10}





Config {256,2,16,10}



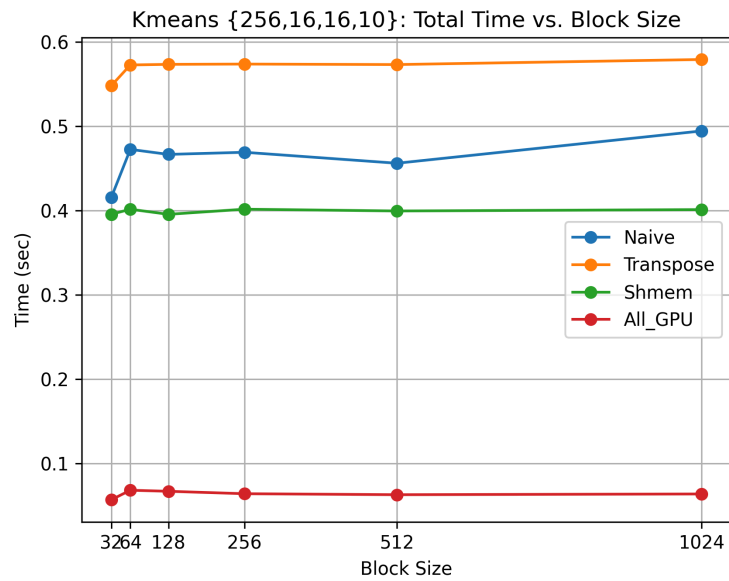
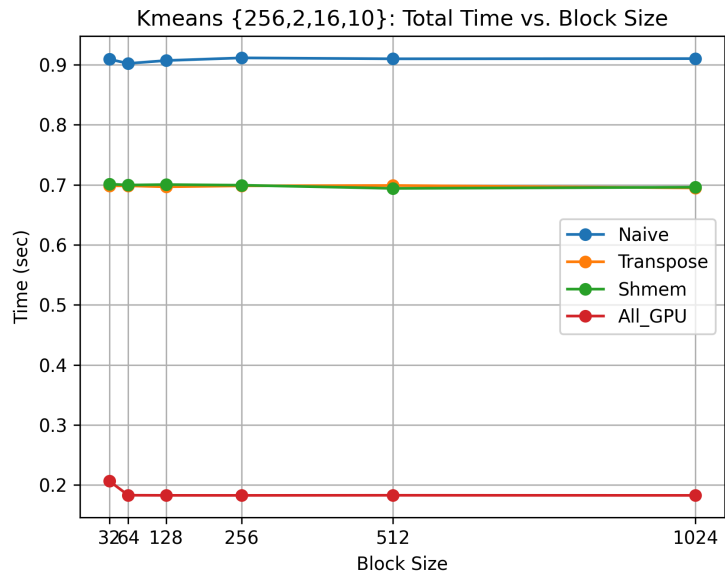


Τοποθετούμε έξτρα time counters κατάλληλα στο κώδικα και αντλούμε τέσσερα επιπλέον χρονικά διαστήματα για να εκτιμήσουμε χρόνο μεταφορών από CPU προς GPU και τανάπαλιν, ενώ μετράμε και χρόνους αμιγώς CPU ή GPU.

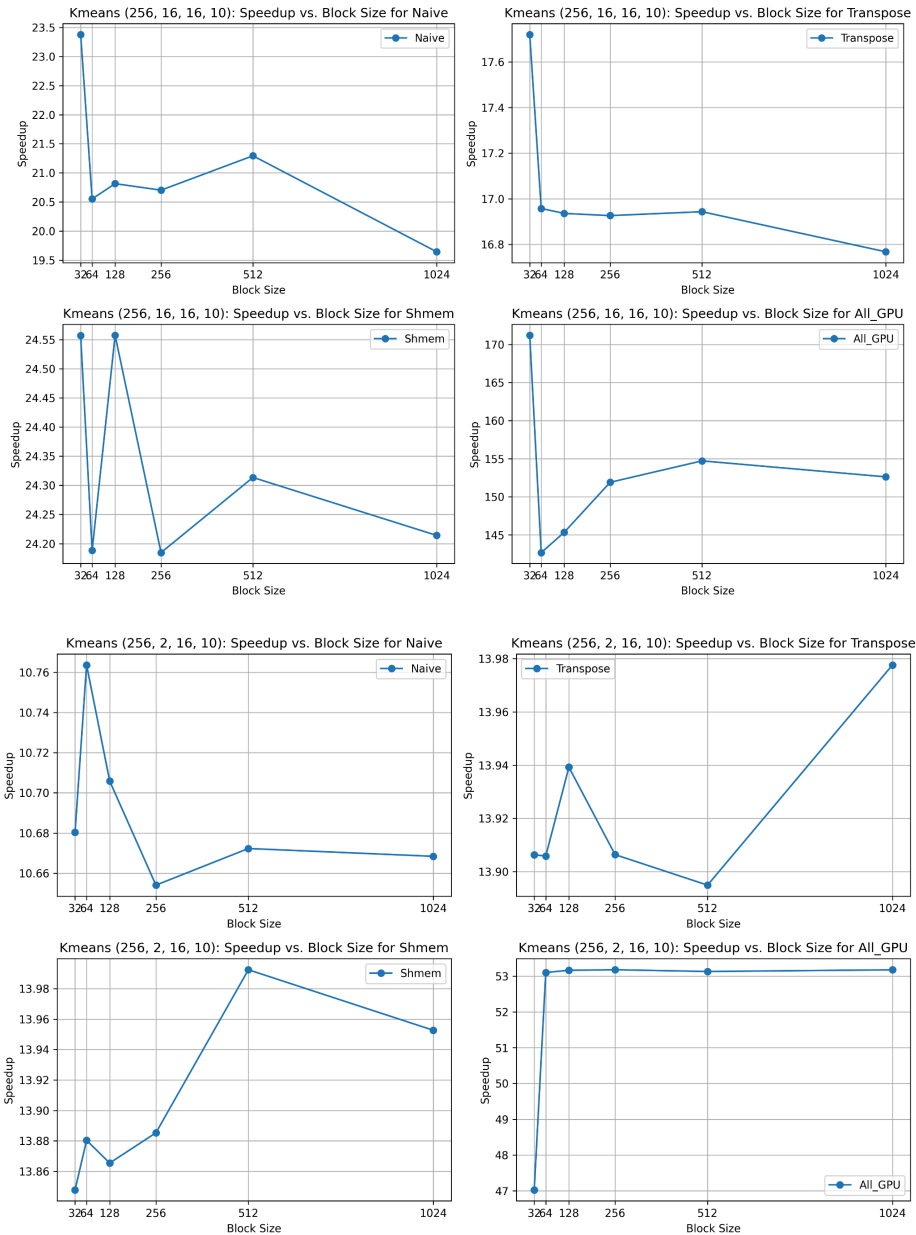
Αφού αντιστρέφουμε τις διαστάσεις των πινάκων, καταστρέφεται η τοπικότητα με συνέπεια να μειώνει την απόδοση της CPU η οποία υποβοηθείται με τις caches της. Συμπεραίνουμε λοιπόν πως το κυρίαρχο bottleneck είναι η CPU.

Είναι εμφανές ότι οι βελτιστοποιημένες υλοποιήσεις υποφέρουν στον χρόνο της CPU και όχι σε αυτόν που δαπανάται στους kernels ή στις μεταφορές, καθώς το μεγαλύτερο μερίδιο χρόνου δαπανάται σε CPU και GPU time.

Bonus ερώτημα: Full-GPU Offload



Speedup Plots for both Configurations:



Σχόλια - Παρατηρήσεις: Όπως επιθυμούμε με βάση το bottleneck των προηγούμενων ερωτημάτων επιλύθηκε η τεράστια καθυστέρηση που επέβαλε η επεξεργασία από τη CPU. Από τα διαγράμματα speedup παρατηρούμε πως σε σχέση με το sequential implementation που πετυχαίνει χρόνο εκτέλεσης 9.71 seconds, τη μεγαλύτερη επιτάχυνση απολαμβάνει το configuration {256,16,16,10}.

Για την επίτευξη του full GPU implementation, έγιναν αρκετές αλλαγές τόσο στον υφιστάμενο πυρήνα (kernel) ενώ προέκυψε και η δημιουργία ενός καινούργιου (update_centroids). Τώρα, ο kernel που καθορίζει σε ποιο cluster ανήκει κάθε αντικείμενο πρέπει επίσης να συγκεντρώνει τις συντεταγμένες του αντικειμένου που ανήκει στο συγκεκριμένο cluster και να αυξάνει το μέγεθός του κατά ένα.

Tail of find_nearest_cluster kernel:

```
103     }
104
105     if (deviceMembership[tid] != index) {
106         /* TODO: Maybe something is missing here... is this write safe? */
107         atomicAdd(devdelta, 1.0);
108         /* (*devdelta) += 1.0; */
109     }
110
111     /* assign the deviceMembership to object objectId */
112     deviceMembership[tid] = index;
113
114
115     /* TODO: additional steps for calculating new centroids in GPU? */
116     // __syncthreads();
117
118     // Atomic operations for updating cluster size and centroid in global memory
119     atomicAdd(&(devicenewClusterSize[index]), 1);
120
121     for(int i = 0; i < numCoords; i++) {
122         atomicAdd(&(devicenewClusters[i * numClusters + index]), deviceobjects[i * numObjs + tid]);
123     }
124
125 }
126
127
```

update_centroids kernel:

```
132
133
134 __global__ static
135 void update_centroids(int numCoords,
136                      int numClusters,
137                      int *devicenewClusterSize,           // [numClusters]
138                      double *devicenewClusters,          // [numCoords][numClusters]
139                      double *deviceClusters)             // [numCoords][numClusters]
140 {
141
142
143     int tid = get_tid();
144     if (tid < numClusters * numCoords) {
145         int coordId = tid % numCoords;
146         int clusterId = tid / numCoords;
147
148         if (devicenewClusterSize[clusterId] > 0)
149             deviceClusters[coordId * numClusters + clusterId] = devicenewClusters[coordId * numClusters + clusterId] /
150             devicenewClusters[coordId * numClusters + clusterId] = 0;
151     }
152     __syncthreads();
153     if (tid < numClusters) devicenewClusterSize[tid] = 0;
154 }
155
156
```

Αυτές οι τιμές θα χρησιμοποιηθούν αργότερα από άλλον kernel (update_centroids) για τον άμεσο υπολογισμό των νέων clusters. Το πρόβλημα είναι ότι οι προσβάσεις στους πίνακες newClusterSize και newClusters πρέπει να γίνονται με atomic instructions και είναι τυχαίες, καθώς κάθε αντικείμενο μπορεί να ανήκει σε οποιοδήποτε cluster. Κάτι τέτοιο πάει κόντρα στην ιδανική δομή μνήμης για μια GPU η οποία θα ήταν συγκεντρωμένη και “στοιχισμένη” (coalesced), επιπρόσθετα τα atomic access καταστρατηγούν τελείως την έννοια της παραλληλοποίησης. Επομένως το update_centroids δεν είναι ιδανικό για GPU. Για να αντιμετωπιστεί αυτό το πρόβλημα, τοποθετήσαμε αυτούς τους πίνακες στην κοινή μνήμη, εξαλείφοντας έτσι το πρόβλημα των μη συγχρονισμένων τυχαίων προσβάσεων, που δεν υπάρχει στην κοινή μνήμη, και μειώνοντας τις συγκρούσεις στις ατομικές εγγραφές.

Το block size φαίνεται να μην επηρεάζει ιδιαίτερα την επίδοση στο config με 2 coordinates. Η διαφορά επίδοσης ανάμεσα σε Coord 2 και Coord 16 config μάλλον οφείλεται στο μεγαλύτερο shared memory size του Coord 16 το οποίο μειώνει την ανάγκη να ανατρέχουμε συχνά στο global memory για να κάνουμε fetch δεδομένα.

To initialization των kernels είναι το ακόλουθο:

```
/* TODO: change invocation if extra parameters needed */
find_nearest_cluster
    <<< numClusterBlocks, numThreadsPerClusterBlock, clusterBlockSharedDataSize >>>
    (numCoords, numObjs, numClusters,
     deviceObjects, devicenewClusters, devicenewClusterSize, deviceClusters, deviceMembership, dev_delta_ptr);
```

```
const unsigned int update_centroids_block_sz = (numCoords* numClusters > blockSize) ? blockSize: numCoords* numClusters; /* TODO:
const unsigned int update_centroids_dim_sz = (numCoords*numClusters + update_centroids_block_sz - 1) / update_centroids_block_sz;
update_centroids<<< update_centroids_dim_sz, update_centroids_block_sz, 0 >>>
    (numCoords, numClusters, devicenewClusterSize, devicenewClusters, deviceClusters);
```

Τέλος, με full GPU implementation έχουμε εξαφανίσει χρόνους μεταφορών host to device και device to host μαζί με το CPU time, εκτός μόνο από το τελικό κομμάτι που η CPU μαζεύει τις μετρήσεις από dimClusters σε clusters.

4.1 Παραλληλοποίηση και βελτιστοποίηση αλγορίθμων σε αρχιτεκτονικές κατανεμημένης μνήμη

K-Means

Συμπληρώνουμε το main.c στο κομμάτι των αναθέσεων των κομματιών του dataset στα επιμέρους ranks ως εξής:

```
142 // Gather membership information from all ranks to tot_membership
143 int recvcunts[size], displs[size];
144 if (rank == 0) {
145     /* TODO: Calculate recvcunts and displs, which will be used to gather data from each rank.
146     * Hint: recvcunts: number of elements received from each rank
147     *       displs: displacement of each rank's data
148     */
149     for (i=0; i<size; i++) {
150         recvcunts[i] = rank_numObjs ;
151         displs[i] = i * rank_numObjs ;
152     }
153 }
154 }
155 }
156
157 /*
158 * TODO: Broadcast the recvcunts and displs arrays to other ranks.
159 */
160 MPI_Bcast(recvcunts, size, MPI_INT, 0, MPI_COMM_WORLD);
161 MPI_Bcast(displs, size, MPI_INT, 0, MPI_COMM_WORLD);
162
163
164 /*
165 * TODO: Gather membership information from every rank. (hint: each rank may send different number of objects)
166 */
167 MPI_Gatherv(membership, rank_numObjs, MPI_INT, tot_membership, recvcunts, displs, MPI_INT, 0, MPI_COMM_WORLD);
168
169
170 if (_debug && rank == 0)
171     for (i = 0; i < numObjs; ++i)
```

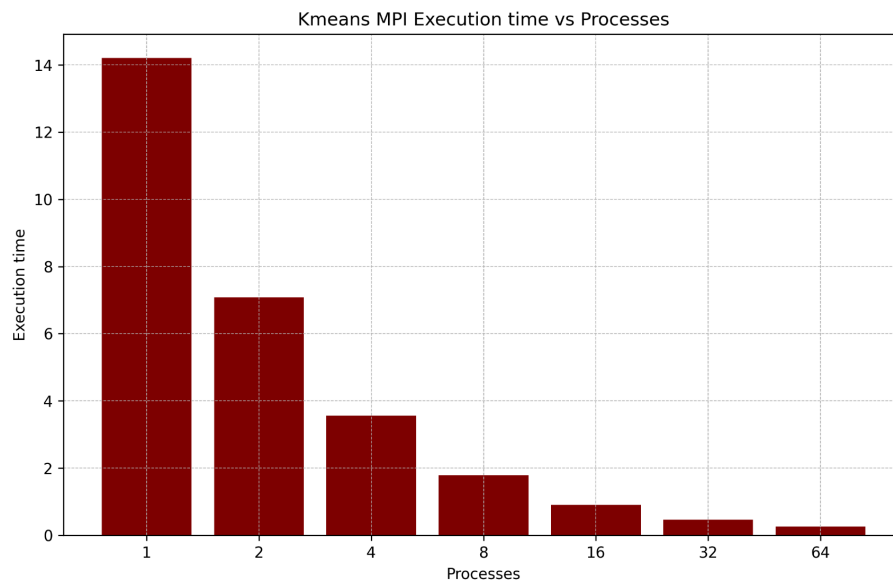
Στο αρχείο kmeans.c το κάθε rank αναλαμβάνει να κάνει τους υπολογισμούς του για τα δικά του objects:

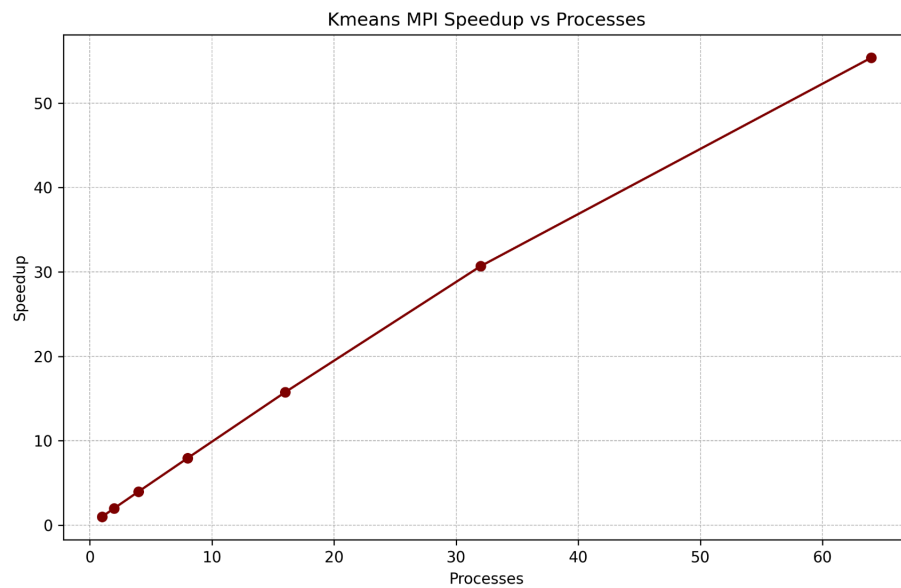
```

100 // update new cluster centers : sum of objects located within
101 rank_newClusterSize[index]++;
102 for (j=0; j<numCoords; j++)
103     rank_newClusters[index*numCoords + j] += objects[i*numCoords + j];
104 }
105
106 /*
107  * TODO: Perform reduction of cluster data (rank_newClusters, rank_newClusterSize) from local arrays to shared.
108  */
109 MPI_Allreduce(rank_newClusters, newClusters, numClusters * numCoords, MPI_DOUBLE, MPI_SUM, MPI_COMM_WORLD);
110 MPI_Allreduce(rank_newClusterSize, newClusterSize, numClusters, MPI_INT, MPI_SUM, MPI_COMM_WORLD);
111
112 // MPI_Reduce(rank_newClusters, newClusters, numClusters * numCoords, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
113 // MPI_Reduce(rank_newClusterSize, newClusterSize, numClusters, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
114
115
116 // average the sum and replace old cluster centers with newClusters
117 for (i=0; i<numClusters; i++) {
118     if (newClusterSize[i] > 0) {
119         for (j=0; j<numCoords; j++) {
120             clusters[i*numCoords + j] = newClusters[i*numCoords + j] / newClusterSize[i];
121         }
122     }
123 }
124
125 /*
126  * TODO: Perform reduction from rank_delta variable to delta variable, that will be used for convergence check.
127  */
128 MPI_Allreduce(&rank_delta, &delta, 1, MPI_DOUBLE, MPI_SUM, MPI_COMM_WORLD);
129
130
131 // Get fraction of objects whose membership changed during this loop. This is used as a convergence criterion.
132 delta /= numObjs;
133

```

{Size, Coords, Clusters, Loops} = {256, 16, 16,10}

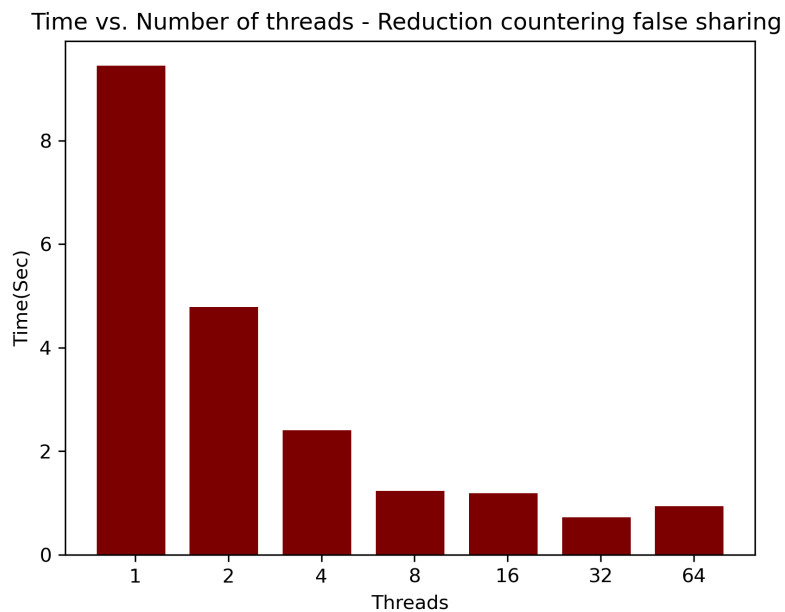


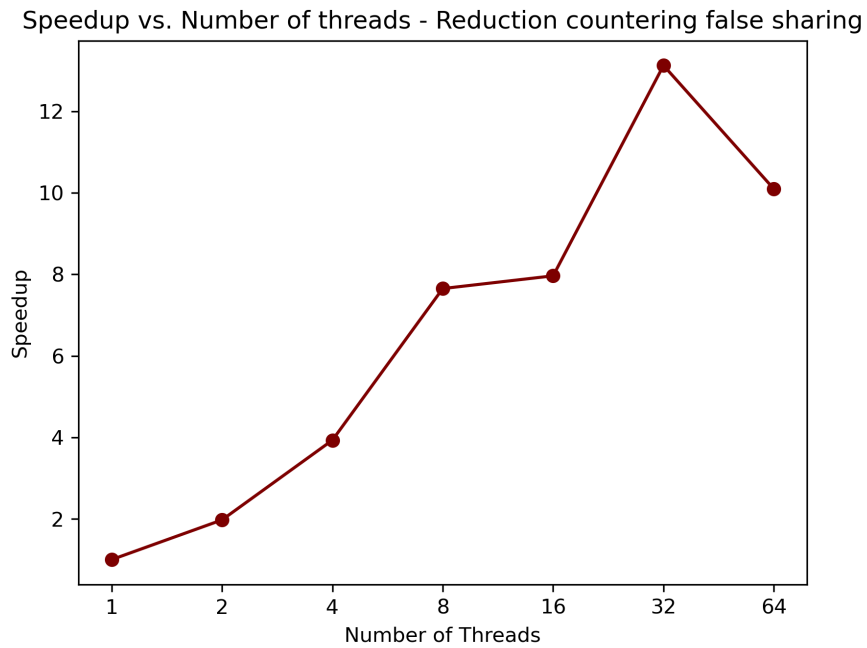


Παρατηρούμε απο τα διαγράμματα χρόνου και speedup σχεδόν τέλεια γραμμική κλιμάκωση καθώς διπλασιάζουμε τα MPI processes.

Bonus ερώτημα:

Η καλύτερη επίδοση που πετύχαμε σε αυτό το config με OpenMP είναι:





Είχαμε υλοποιήσει σχήμα reduction πηγαίνοντας να προσομοιώσουμε κάπως αυτό που γίνεται και στην MPI με distributed processing. Αυτό έγινε λόγω της διαίρει και βασίλευε λογικής του reduction.

Παρατηρούμε πως πραγματικά γραμμικό speedup φαίνεται να εμφανίζει η MPI ενώ η OpenMP λόγω thread communication και resource conflicts στη CPU και ιδιαίτερα στο πεδίο του hyperthreading (64 threads) χάνει κλιμάκωση. Στη πράξη βλέπουμε ότι πραγματικό reduction πραγματοποιείται στο MPI που τα processes γίνονται σε ξεχωριστά nodes του parlab.

Heat transfer

Μετρήσεις με έλεγχο σύγκλισης $T=256$ σε πίνακα 1024×1024

Μέθοδος	TotalTime
Jacobi	221.430868
Gauss-Seidel SOR	2.201772
RedBlack SOR	1.395950

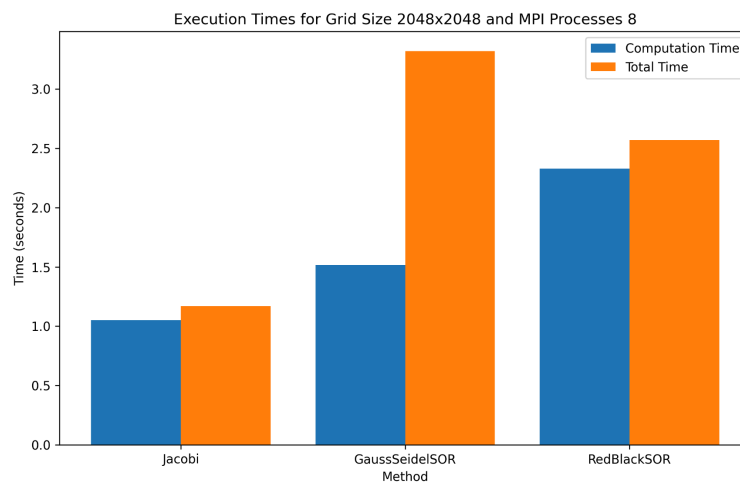
Παρατηρούμε τεράστια διαφορά στους χρόνους σύγκλισης για τις βελτιωμένες μορφές του αλγορίθμου (δηλαδή GaussSeidelSOR και RedBlackSOR). Εξηγείται η επιτάχυνση αυτή και στο πίνακα αριθμού iterations:

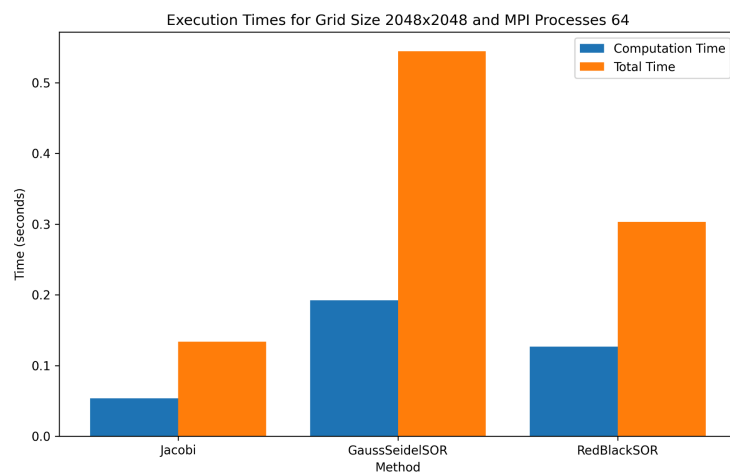
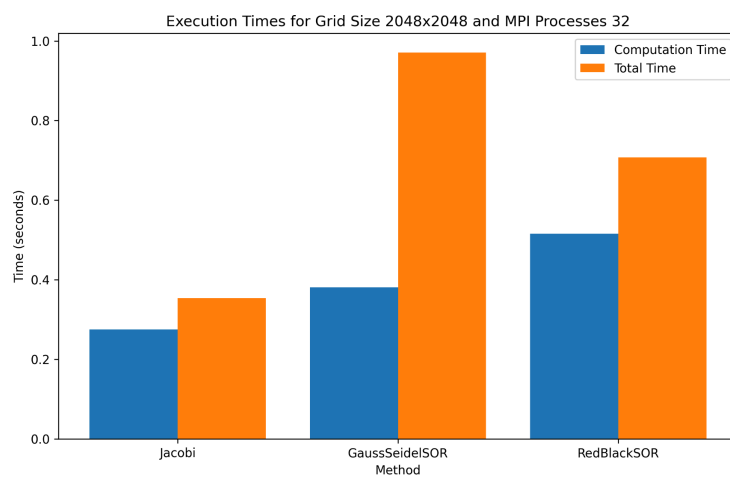
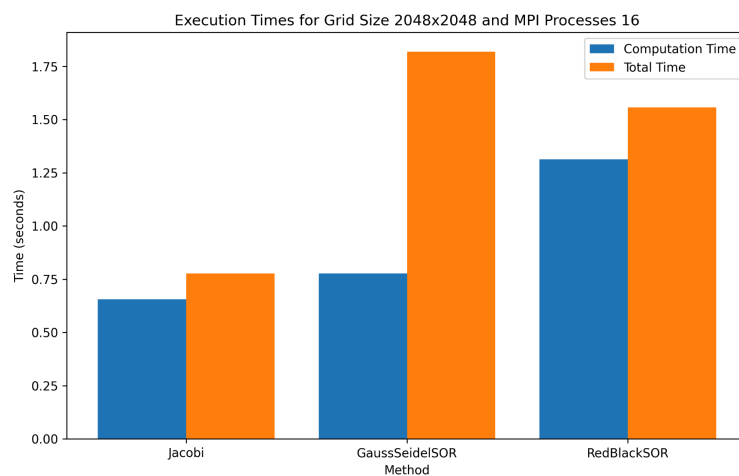
Μέθοδος	Iterations
Jacobi	798201
Gauss-Seidel SOR	3201
RedBlack SOR	2501

Παρολο που υπολογιστικά οι τροποποιήσεις δε φαίνεται να έχουν ουσιαστική διαφορά, μελετώντας το κώδικα, επιτυγχάνουν σύγκλιση σε λιγότερα iterations και αυτό αποτυπώνεται στο συνολικό χρόνο.

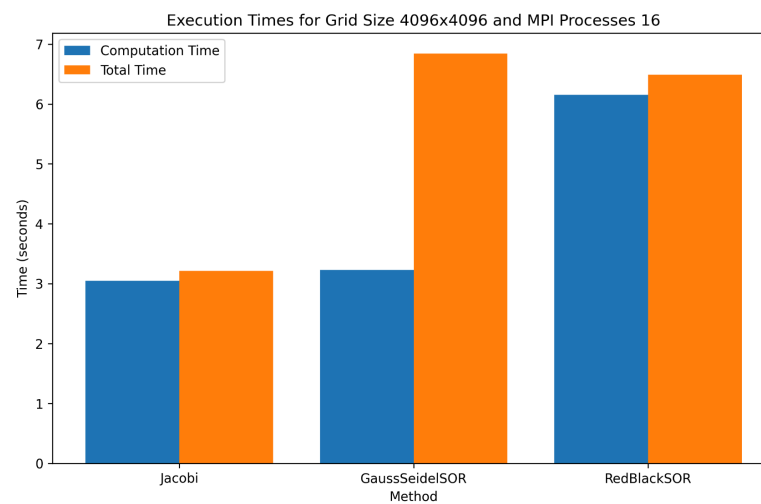
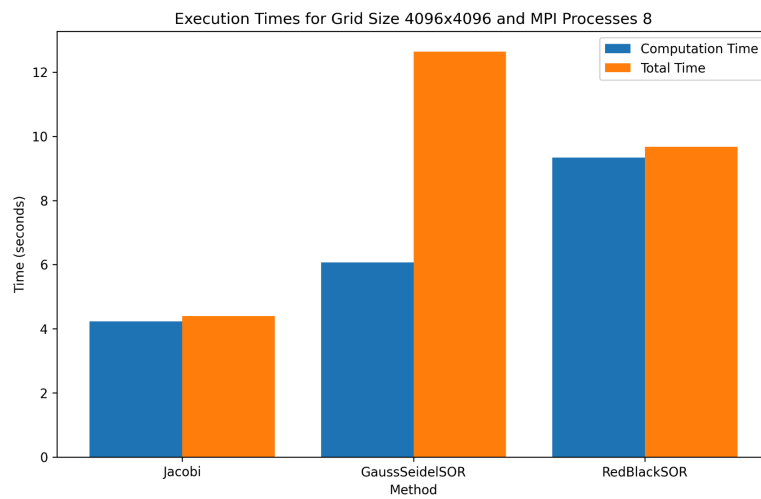
Μετρήσεις χωρίς έλεγχο σύγκλισης

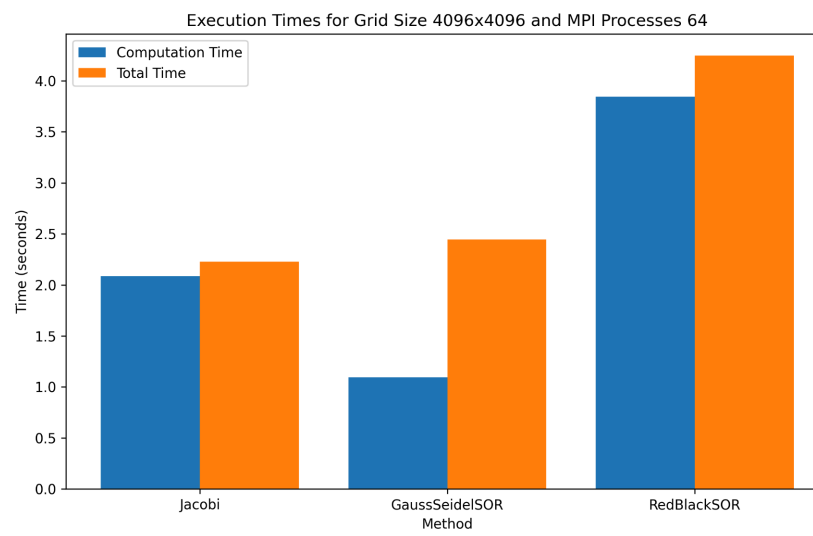
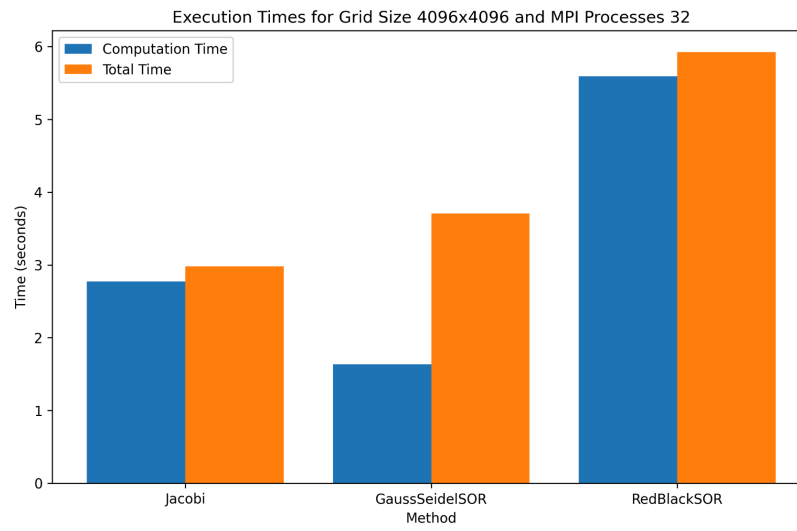
Πίνακας 2048x2048



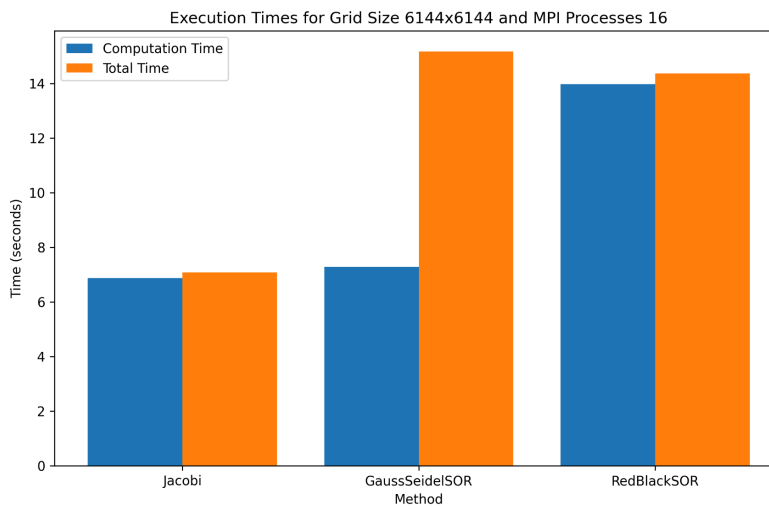
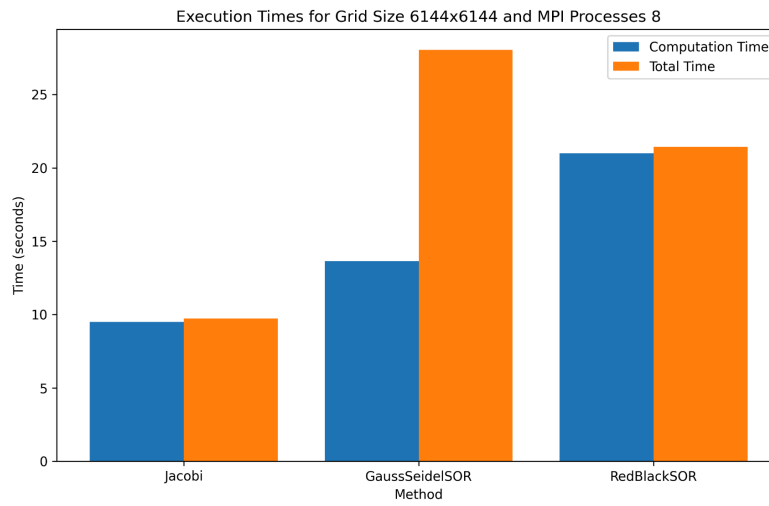


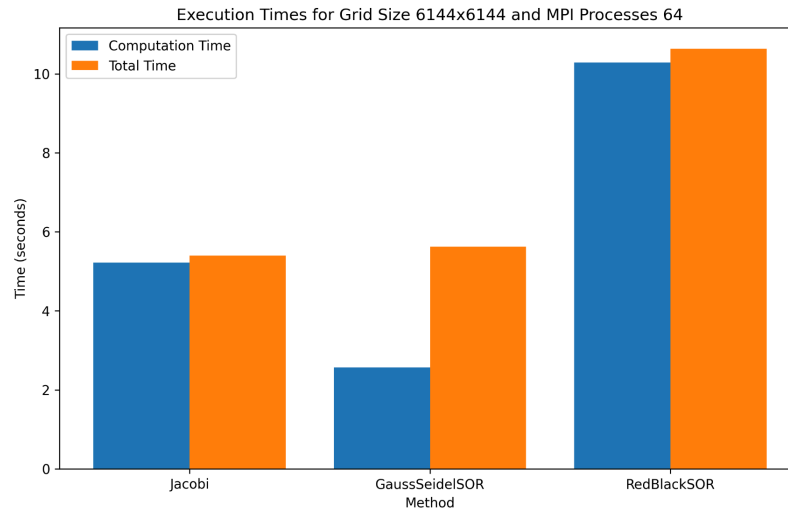
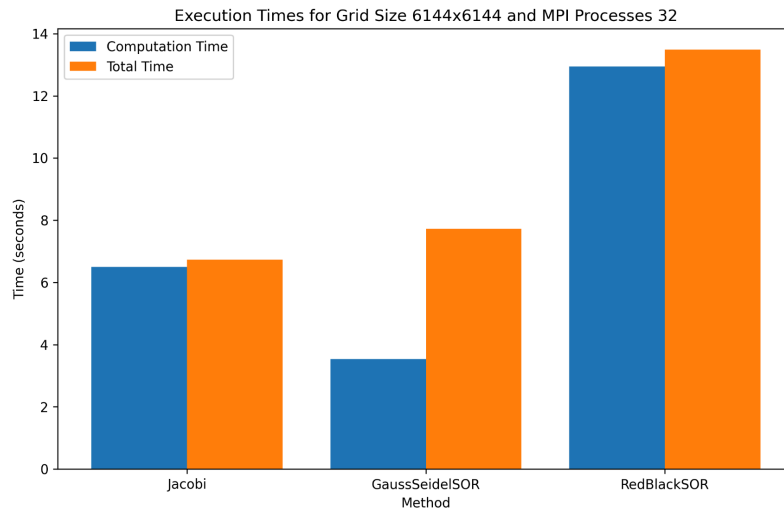
Πίνακας 4096x4096





Πίνακας 6144x6144





Ποιοτική ανάλυση και ερμηνεία διαγραμμάτων

Jacobi: Σε κάθε επανάληψη όλες οι διεργασίες συλλέγουν αλλα και αποστέλλουν δεδομένα με γειτονικές διεργασίες ύστερα πραγματοποιούν τους τοπικούς τους υπολογισμούς. Όλες οι διεργασίες χρονικά συμμετρικές χωρίς εξαρτήσεις δεδομένων ίδιας χρονικής στιγμής κατα των υπολογισμο των νέων τιμων των σημείων.

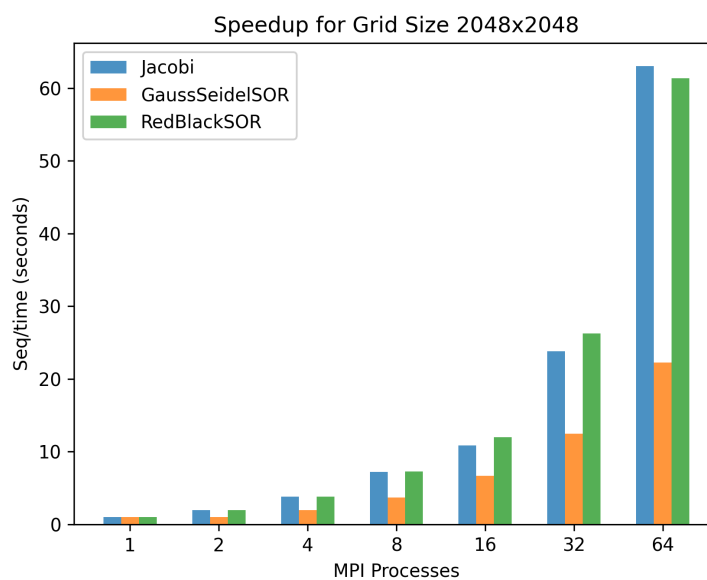
Gauss Seidel: Δεδομένης της εξάρτησης κάθε στοιχείου του πίνακα από το πιο αριστερό και το από πάνω του, δεν έχουμε συμμετρική επικοινωνία κατά τη διάρκεια κάθε επανάληψης. Ειδικότερα, απαιτείται η πρώτη διεργασία στην επάνω αριστερή γωνία να ολοκληρώσει πρώτα τους υπολογισμούς της και να μεταδώσει τα στοιχεία της προς τα δεξιά και κάτω. Το ίδιο μοτίβο

πρέπει να ακολουθήσει και κάθε επόμενη διεργασία, μεταδίδοντας τα αποτελέσματά της δεξιά και κάτω, αφού ολοκληρώσει τους υπολογισμούς της. Ωστόσο, μια διεργασία δεν μπορεί να αρχίσει πριν λάβει τα δεδομένα του τρέχοντος χρονικού βήματος από την αριστερή και την από πάνω της διεργασία. Αυτή η αλυσιδωτή εξάρτηση μεταξύ των διεργασιών οδηγεί την κάτω δεξιά διεργασία να περιμένει σημαντικά για τα δεδομένα, κάνοντας το κάθε επανάληψη να διαρκεί περισσότερο σε σύγκριση με άλλες μεθόδους.

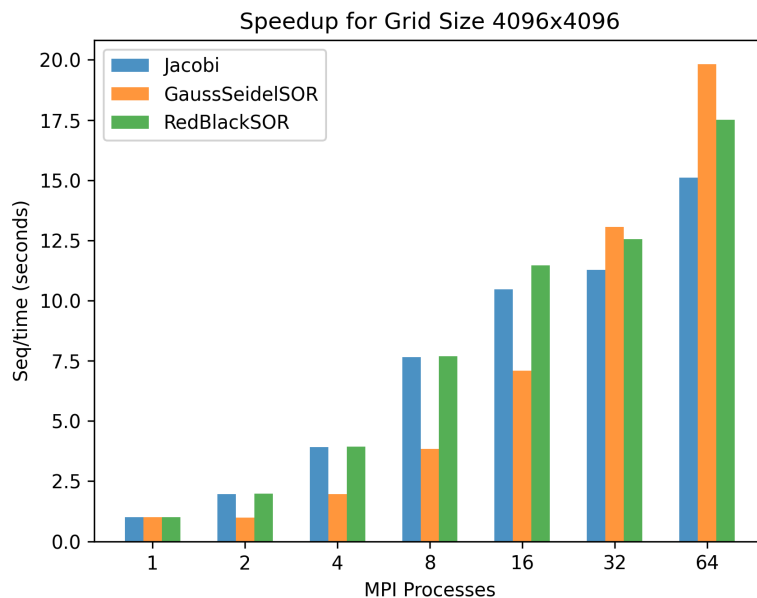
RedBlack SOR: Στη κόκκινη φάση θέλουμε δεδομένα από τη προηγούμενη επανάληψη οπότε πρώτα τα στέλνουμε από τις υπόλοιπες διεργασίες και μετά υπολογίζουμε, ύστερα στη μαύρη φάση υπολογίζουμε βάση των προυπολογισμένων μαύρων κελιών, δεν παρατηρούμε κάποια ασυμμετρία.

Speedup για 2048,4096,6144 χωρίς έλεγχο σύγκλισης

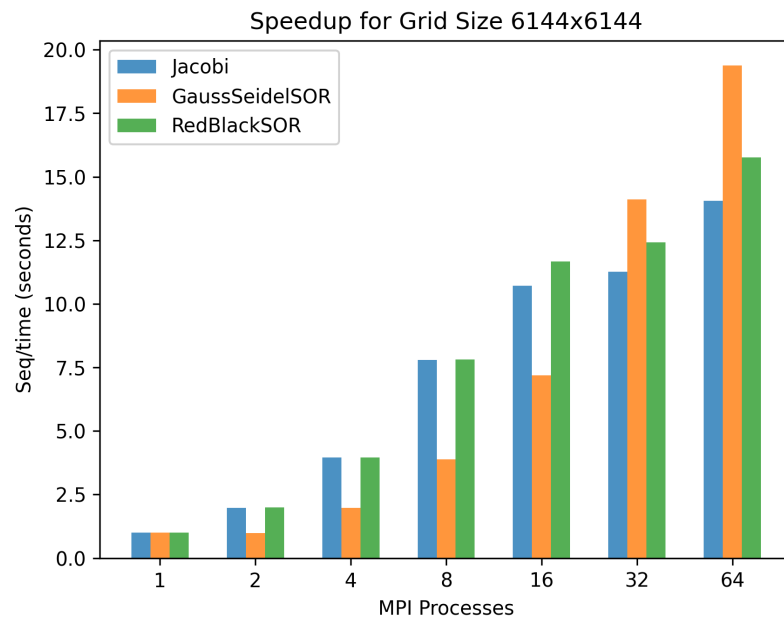
Η έννοια “time” αφορά το totaltime στα παρακάτω διαγράμματα



Εξετάζοντας το παραπάνω διάγραμμα παρατηρούμε ότι έχουμε κανονικό scaling καθώς αυξάνουμε τα threads, η μέθοδος Gauss-Seidel δυσκολεύεται να αποκτήσει μεγάλο speedup λόγω των εξαρτήσεων που αναλύσαμε και πιο πάνω. Jacobi και RedBlack φαίνεται να έχουν παρόμοια συμπεριφορά.



Για 4096x4096 παρατηρούμε ένα ήπιο σταμάτημα στο scaling για Jacobi και RedBlackSOR για πάνω από 16 threads. Αυτό οφείλεται στο cache contention των CPU. Στο parlab έχουμε συνολικά $8 \times 2 = 16$ cores για υπολογισμό. Αν πάμε να μεγαλώσουμε τα threads μπαίνουμε στα νερά του hyperthreading κάτι που αυξάνει τα conflicts στη cache. Παρατηρούμε ότι στον Gauss-Seidel δεν υπάρχει τέτοια συμπεριφορά και οφείλεται πιθανότατα ότι έχει μειωμένο cache contention λόγω οι υπολογισμοί των κελιών δε πραγματοποιούνται ακριβώς συγχρονισμένα, λόγω αλυσιδωτής σχέσης.



Ίδια συμπεριφορά παρατηρούμε και για 6144x6144. Μπορούμε να το ερμηνεύσουμε υπο το ίδιο πρίσμα όπως και για τη περίπτωση 4096x4096.